VeRedact-PQ: Scalable Post-Quantum Redaction with Multi-Party Authorization and Verifiable Auditing

Rangsimann Sattayarom, Ratchanon Wongwitutai, Natthawat Tungsriworakan,
Tagrid Chongkolrattanapond, Somchart Fugkeaw
Sirindhorn International Institute of Technology, Thammasat University, Thailand
6622771747@g.siit.tu.ac.th, 6622780268@g.siit.tu.ac.th, 6622772364@g.siit.tu.ac.th, 6622781175@g.siit.tu.ac.th,
somchart@siit.tu.ac.th

***Abstract*—Redactable permissioned blockchains enable legitimate modification of committed transactions, but existing approaches face challenges in distributed authorization, privacy-preserving accountability, post-quantum security, and scalability under high redaction workloads. In particular, independently validating, authorizing, executing, and auditing each redaction can incur substantial cryptographic and committee overhead, while controlled modification must remain bound to the applicable policy and current ledger state. We propose *VeRedact-PQ*, a post-quantum authenticated and verifiable redaction framework for permissioned blockchains. VeRedact-PQ separates immutable transaction-policy bindings from redactable content and employs post-quantum authentication and zero-knowledge verification for privacy-preserving request validation. A workload-aware adaptive batching mechanism amortizes multi-party committee authorization while preserving request-level policy enforcement. For efficient execution, authorized modifications targeting the same transaction batch are coalesced into shared Merkle updates, requiring only one distributed post-quantum chameleon-hash adaptation per resulting batch-root transition. Cryptographic evidence binds request validation and committee authorization to the executed redaction, providing individual accountability despite batched processing. Finally, a sharded authenticated audit index supports privacy-preserving query-based verification using compact multiproofs and shared batch evidence, while expensive post-quantum zero-knowledge verification is reserved for challenged or forensic audits. VeRedact-PQ thereby provides an end-to-end post-quantum redaction workflow that amortizes authorization, authenticated-state update, redaction, and audit-verification costs while maintaining policy binding, distributed control, privacy, and verifiable accountability.**

***Index Terms*—Permissioned Blockchain, Redactable Blockchain, Post-Quantum Chameleon Hash, Post-Quantum Security, Verifiable Redaction, Privacy-Preserving Auditing, Large-scale Transaction Processing.**

I. Introduction

Permissioned blockchains provide a shared and auditable ledger for multi-organization environments without requiring complete mutual trust [7]. Their integrity fundamentally relies on cryptographic immutability: once a transaction is committed, unauthorized modification becomes detectable. Strict immutability, however, can conflict with legitimate enterprise requirements, including correction of erroneous records, removal of sensitive information for privacy or regulatory compliance, and controlled update of obsolete data [9], [21], [22] , [38]. Supporting such operations without undermining ledger integrity creates the fundamental problem of *controlled and accountable blockchain redaction*.

Redactable blockchains commonly employ chameleon hashes to modify committed data while preserving blockchain consistency [1], [2], [6], [30]. Subsequent studies have strengthened this model through fine-grained access control [8], [15], decentralized trapdoor management [3], [19], dynamic updates [5], [16], and threshold redaction [27]. Nevertheless, preserving the ledger commitment alone does not establish that a requester is authorized for a particular transaction, operation, policy, and current state. Moreover, because an authorized chameleon-hash collision preserves blockchain linkage, conventional ledger verification cannot by itself establish why a modification was permitted or whether the required authorization process was followed. Recent policy-based, traceable, and publicly auditable constructions improve accountability [33]–[35], but integrating request-level policy validation with distributed authorization and verifiable execution remains important for multi-organization redaction.

Privacy and long-term security further complicate this problem. Fine-grained and policy-hiding schemes can restrict redaction authority or conceal authorization policies [4], [18], [31], [34], while verifiable and auditable designs provide mechanisms for checking redaction integrity [13], [20], [33]. However, authorization may depend on sensitive credentials, roles, or policy attributes that should not be disclosed merely to prove eligibility, and auditors should not need access to the underlying sensitive transaction to verify a redaction. In parallel, lattice-based and quantum-resistant chameleon hashes have been investigated for redactable blockchains [10], [11], [17], [37]. Protecting only the chameleon-hash primitive, however, is insufficient for an end-to-end post-quantum workflow if requester authentication, private authorization, committee approval, or audit authentication still depends on classical cryptography.

Scalability is another major challenge. Existing studies have addressed scalable redaction [14], [21], [26], efficient data-structure updates [23], lightweight storage and verification [20], and robust threshold redaction [27]. Nevertheless, under high transaction and redaction volumes, ledger-wide target lookup, per-request policy verification, individual commit-

tee authorization, repeated Merkle updates, and independent chameleon-hash adaptations can collectively incur substantial overhead. Fixed or request-by-request processing also cannot adapt well to varying workloads, while batching must preserve individual policy enforcement so that an invalid request cannot inherit the authorization of valid requests. Similarly, independently verifying every redaction record can make large-scale auditing expensive. Efficient redactability therefore requires coordinated scalability across *request resolution, authorization, redaction execution, and auditing*, while retaining request-level accountability.

To address these challenges, we propose *VeRedact-PQ*, a post-quantum authenticated, scalable, and verifiable redaction framework for permissioned blockchains. VeRedact-PQ provides an end-to-end redaction workflow that separates request validation, multi-party authorization, controlled state modification, and privacy-preserving auditing. Rather than applying expensive cryptographic and committee operations independently to every request, the framework combines request-level policy enforcement with adaptive batch authorization, coalesced state updates, and query-scoped audit verification. Post-quantum mechanisms protect the security-critical authentication, private authorization, redaction, and audit paths, while compact attestations and authenticated-data-structure proofs avoid unnecessary repeated PQ processing. The resulting design preserves individual accountability from redaction request to auditable state transition while amortizing common operations under high-volume workloads. The main contributions are summarized as follows:

- **Post-Quantum Authenticated and Policy-Bound Redaction:** We design an end-to-end post-quantum redaction workflow comprising PQ requester authentication, Post-Quantum Zero-Knowledge (PQZK) private policy verification, t-of-n PQ committee authorization, and distributed Post-Quantum Chameleon Hash (PQCH) redaction. The proposed Redaction-Aware Dual Commitment (RADC) separates the immutable transaction-policy binding from the redactable content state, ensuring that an authorized content modification cannot alter its protected policy context.
- **Scalable Resolution and Adaptive Authorization:** We develop a sharded authenticated lookup structure for efficient redaction-target resolution and an Adaptive Batch Round Redaction Request (ABRRR) mechanism for workload-aware multi-party authorization. Requests are independently validated before batching and represented by compact PQ-signed validation attestations, enabling batch authorization without repeating expensive PQZK verification.
- **Coalesced and Verifiable Batch Redaction:** We develop a Batch Incremental Merkle Commitment (BIMC) mechanism that coalesces authorized modifications within each transaction batch, allowing them to share Merkle authentication paths and incur only one distributed PQCH adaptation for the resulting batch-root transition. The Policy-Bound Batch Redaction Proof (PBRP) cryptographically binds request-level validation and committee authorization to the executed state transition, preserving individual verifiability under batched redaction.
- **Cost-Aware Post-Quantum Auditing:** We design a sharded Redaction Audit Index (RAI) for privacy-preserving verification of redaction integrity, history, authorization, policy compliance, scope, and freshness. Query-scoped Merkle multiproofs, shared batch evidence, and query-adaptive proof disclosure reduce routine verification and communication costs, while complete PQZK verification remains available for challenged or deep forensic audits.

II. Related Work

A. Distributed and Scalable Blockchain Redaction

Redactable blockchains commonly employ chameleon hashes to modify committed data while preserving blockchain consistency [9]. Subsequent studies extended this model to consortium blockchains [2] and decentralized redaction through distributed chameleon hashes and trapdoor management [1], [3], [6], [19], [30]. Multi-party chameleon hashing [12] and robust threshold redaction [27] further reduce dependence on a single redaction authority, while dynamic and controlled constructions support evolving redaction requirements [5], [16], [36].

Scalability has also been addressed through efficient update mechanisms, lightweight storage, specialized Merkle structures, and application-oriented redaction [14], [20], [21], [23], [26]. However, efficiency is typically optimized at a particular layer, whereas high-volume redaction incurs combined costs from target lookup, request validation, committee authorization, authenticated state updates, and chameleon-hash adaptation. VeRedact-PQ addresses these costs jointly through sharded lookup, adaptive request batching, coalesced Merkle updates, and one distributed PQCH adaptation per affected transaction-batch root transition.

B. Policy-Controlled, Private, and Verifiable Redaction

Fine-grained redaction has been developed using attribute- and policy-based mechanisms [8], [15], while privacy-preserving and policy-hiding constructions restrict disclosure of sensitive authorization information [4], [18], [24], [31], [34]. Other schemes strengthen traceability and accountability through dynamic policies, anonymous but accountable redaction, policy-compliant rewriting, and cross-chain accountability [25], [28], [29], [32], [35]. Verifiable redaction has likewise been studied through efficient query and integrity auditing [13], lightweight verification and permission supervision [20], and publicly auditable architectures [33].

Despite these advances, policy validation, multi-party authorization, redaction execution, and auditing are often treated as separate functions. VeRedact-PQ instead binds them at request level: private policy eligibility is verified before batching, committee authorization is bound to an authenticated batch, and PBRP links this evidence to the executed state transition. A sharded RAI subsequently supports query-based auditing using compact multiproofs and shared batch evidence, preserving individual accountability without repeatedly processing complete redaction proofs.

TABLE I: Comparison of VeRedact-PQ with Representative Redactable Blockchain Schemes

Scheme	PQ Security	Distributed/ Threshold Auth.	Policy Control	Scalable/ Batch Redaction	Private Verification	Verifiable Auditing
Scheme [1]	×	✓	×	×	×	△
Scheme [5]	×	△	✓	△	×	✓
Scheme [13]	×	×	×	×	×	✓
Scheme [14]	×	×	△	✓	✓	×
Scheme [15]	×	✓	✓	×	△	△
Scheme [17]	✓	×	△	×	×	×
Scheme [20]	×	△	✓	△	△	✓
Scheme [27]	×	✓	×	×	×	×
Scheme [33]	×	△	✓	△	△	✓
Scheme [34]	×	✓	✓	×	✓	×
VeRedact-PQ	✓	✓	✓	✓	✓	✓

✓: Supported; △: Partially supported; ×: Not supported

C. Post-Quantum Redactable Blockchains

Post-quantum redactable blockchain research has primarily focused on the chameleon-hash primitive. Existing approaches include quantum-resistant key-exposure-free chameleon hashing [11], lattice-based redactable blockchains [10], quantum-resistant redaction with trapdoor updates [17], and lattice-based policy- or identity-oriented chameleon hashes [18], [37]. These works provide important foundations for protecting controlled redaction against quantum adversaries.

However, a quantum-resistant chameleon hash alone does not provide an end-to-end post-quantum redaction workflow if authentication, authorization, committee approval, or auditing still relies on classical mechanisms. VeRedact-PQ therefore extends PQ protection across the security-critical workflow using PQ authentication, PQZK-based private policy validation, PQ multi-party authorization, and distributed PQCH redaction. To control overhead, expensive PQZK verification is performed only when required, while compact PQ-signed attestations and hash-based authenticated proofs support subsequent authorization and routine auditing.

Overall, VeRedact-PQ differs from existing work by jointly addressing *post-quantum security, scalable multi-party redaction, and privacy-preserving verifiability* across the complete redaction lifecycle rather than optimizing these properties independently.

As summarized in Table I, existing redactable blockchain schemes typically address only a subset of the requirements for secure and scalable enterprise redaction. Decentralized and threshold-based approaches strengthen distributed control [1], [15], [27], while policy-oriented schemes provide fine-grained or privacy-aware redaction authorization [5], [34]. Other works primarily improve scalability [14], [20] or verifiable auditing [13], [33]. Quantum-resistant schemes such as [17] protect the underlying redaction primitive, but do not jointly provide post-quantum authentication, private policy validation, distributed authorization, and audit verification. In contrast, VeRedact-PQ provides more comprehensive properties of the redaction lifecycle. It supports request-level PQ authentication and policy verification with distributed batch authorization, coalesced redaction execution, and privacy-preserving auditing. Importantly, this integration is cost-aware: expensive PQ verification is not repeatedly invoked across all stages, while authenticated batching, shared state updates, and query-scoped proofs amortize authorization, redaction, and auditing costs under high-volume workloads.

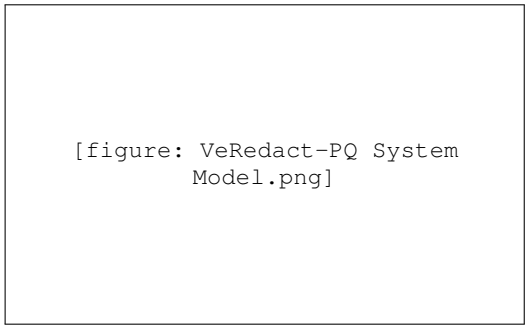


Fig. 1: System model of the proposed VeRedact-PQ framework

III. Our Proposed VeRedact-PQ Scheme

This section presents the system model, threat model, and system process of our proposed VeRedact-PQ scheme.

A. System Model

As illustrated in Fig. 1, VeRedact-PQ considers a permissioned blockchain operated by multiple mutually accountable organizations. The blockchain stores enterprise transactions whose contents may subsequently require legitimate modification or removal according to predefined redaction policies. Rather than assigning the redaction capability to a single privileged authority, VeRedact-PQ distributes redaction authorization among an epoch-based committee and maintains cryptographic evidence that allows an authorized redaction to be independently verified. The system consists of the following entities.

1) Data Owner (DO): A data owner generates and submits transactions to the permissioned blockchain. Before submission, the DO signs the transaction to provide origin authentication and integrity. Each transaction is associated with its identifier, data type, timestamp, and applicable redaction policy. When permitted by the corresponding policy, the DO may also submit a redaction request for previously committed data.

2) Authorized Requester (AR): An authorized requester is an authenticated entity permitted to request modification or removal of a committed transaction. The requester submits a signed redaction request specifying the target transaction and requested modification. It also provides the information required to demonstrate compliance with the applicable redaction policy. The requester may coincide with the original data owner or may be another entity granted redaction rights by the consortium.

3) Redaction Committee (RC): The redaction committee consists of authorized consortium members that collectively govern the redaction capability. Committee membership is defined for an authorization epoch e, allowing the authorization structure to evolve over time. The committee members jointly participate in the distributed management of the post-quantum chameleon-hash trapdoor and authorize eligible redactions through threshold approval. Consequently, no individual committee member is intended to exercise the complete redaction capability independently.

4) Validation and Policy Service (VPS): The VPS validates incoming redaction requests before they become eligible for committee authorization. It authenticates the requester, retrieves the policy associated with the target transaction, verifies the requested redaction against the permitted scope and validity conditions, and verifies the corresponding policy-compliance zero-knowledge proof. Only successfully validated requests are admitted to the subsequent authorization process.

5) Permissioned Blockchain Network (PBN): The PBN is maintained by authenticated consortium nodes executing the underlying permissioned consensus protocol. It records transaction commitments, policy references, epoch information, redaction checkpoints, and verification metadata. Transaction batches are committed through Merkle roots and a post-quantum chameleon hash (PQCH), allowing an authorized modification to preserve the required ledger linkage while unauthorized modifications remain detectable. The blockchain additionally provides an immutable reference for redaction authorization and subsequent auditing. The PBN also hosts two service functions rather than separate entities: a *checkpoint authority*, operated by the PBN ordering service, which signs authenticated batch checkpoints with sk_{CP}, and an *audit service*, which maintains the Redaction Audit Index (RAI), resolves authenticated auditor queries, and signs audit responses with sk_A. Neither function needs to be trusted for integrity, since their outputs are verified against blockchain-anchored roots and checkpoints. Large supporting objects, including complete PQZK proofs, complete PBRP objects, and index entries, are kept in an off-ledger evidence store maintained by the PBN nodes; their integrity is guaranteed by the digests and authenticated roots anchored on-chain.

6) Auditor (AU): An auditor is an authorized internal or external verifier responsible for examining the legitimacy of completed redactions. The auditor does not require access to the complete sensitive transaction content. Instead, it verifies the corresponding policy-bound redaction evidence, authenticated batch membership, committee authorization, and anchored blockchain state to determine whether the redaction was properly authorized and executed.

For scalable redaction processing, successfully validated requests are placed in a pending request queue $\mathcal{Q}_e$ maintained by the VPS which issues a signed admission receipt for every queued request so that an admitted but unprocessed request is attributable. VeRedact-PQ employs the *Adaptive Batch Round Redaction Request (ABRRR)* mechanism to dynamically form a batch $\mathcal{B}_e$ according to the observed request rate, queue state, and maximum waiting time. Each request remains individually policy validated before batching. The corresponding *Policy-Bound Batch Redaction Proof (PBRP)* cryptographically binds every validated request to its target transaction, policy state, and authorization epoch. The committee subsequently produces threshold authorization over the authenticated batch commitment, allowing common authorization and consensus operations to be amortized while retaining request-level accountability.

Accordingly, VeRedact-PQ separates three security responsibilities: *request-level validation*, performed before batching; *distributed redaction authorization*, performed by the epoch committee; and *post-redaction verification*, performed using PBRP and the blockchain-anchored state. This separation prevents an invalid request from inheriting the authorization of other requests in the same batch while enabling efficient and independently verifiable redaction in large-scale permissioned blockchain environments.

B. Threat Model

VeRedact-PQ considers malicious or compromised participants in a permissioned blockchain. The main threats and security assumptions are defined as follows.

1) Malicious Requester: A malicious or compromised requester may submit unauthorized, malformed, or replayed redaction requests, or attempt to modify data outside the permitted policy scope. Each request must therefore be authenticated and independently validated against its associated redaction policy before authorization.

2) Malicious Committee Members: A subset of redaction committee members may collude to perform an unauthorized redaction or misuse their PQCH trapdoor shares. We assume that fewer than the required threshold of committee members are compromised; therefore, they cannot independently exercise the distributed redaction capability or generate valid threshold authorization.

3) Malicious Blockchain Participant: A compromised consortium node may attempt to alter, remove, or inject transaction or redaction information, including a node operating the audit service that returns altered or stale audit responses. VeRedact-PQ assumes that the underlying permissioned blockchain satisfies its prescribed consensus fault-tolerance condition.

TABLE II: Summary of Major Notations

Notation	Description	Notation	Description
λ, PP	Security parameter, public parameters	R_i, σ_i^R	Redaction request, requester signature
e, e_P, e_i	Authorization, policy, and state epochs	x_i, w_i, π_i^{PQ}	PQZK statement, witness, proof
(pk_i, sk_i)	PQ signature key pair of entity i	C_i^{VR}, α_i	Validated-request commitment, VPS attestation
sk_V, sk_{CP}, sk_A	VPS, checkpoint, audit-service keys	VR_i, $\mathcal{Q}_e$	Validated request, pending queue
K_{idx}, K_A	PRF keys for SA-RLI and RAI	$\mathcal{B}_e$, B_e^*	ABRRR batch, target batch size
$\mathcal{C}_e$, n, t	Committee, size, threshold	η_i, R_e^{VR}	Request leaf, validation root
pk_{CH}, $td_{e,k}$	PQCH public key, trapdoor share	C_e^B, $Auth_e^B$	Batch commitment, authorization evidence
P_j, PID_j, v_j, C_{P_j}	Policy, identifier, version, commitment	$\sigma_{e,k}^B$, $\mathcal{A}_e$	Committee approval, approval set
m_i, ρ_i	Transaction content, commitment randomness	$\delta_{b,k}$	PQCH adaptation share
TID_i, DT_i, ts_i	Identifier, data type, timestamp	MP_b^{multi}	BIMC multiproof
σ_i, C_i^{orig}	Data-owner signature, provenance commitment	$PBRP_i^{auth}$, $PBRP_i$	Authorization and final redaction proof
I_i, D_i, L_i	RADC binding, content commitment, leaf	RR_i	Redaction record
MR_b, r_b, CH_b	Batch root, randomness, chameleon hash	E_i^A, R_{RAI}, CP_e^A	Audit entry, RAI root, audit checkpoint
v_b, A_b	Batch version, batch checkpoint	Q_j^A, $Resp_j^A$	Audit query, audit response
τ_i, E_i	SA-RLI token, index entry	R_{RLI}, v_{RLI}	SA-RLI root, snapshot identifier
X_i^{red}	Redaction payload (m_i', ρ_i')	D_i'	Proposed content commitment
rc_i	VPS admission receipt		

4) Batch-Manipulation Adversary: An adversary may attempt to insert an invalid request into an authorized batch, replace a validated request, or reuse authorization evidence. PBRP binds each validated request to its target transaction, policy state, and authorization epoch, making such manipulation detectable.

5) Privacy-Curious Auditor: An auditor or consortium participant may attempt to infer sensitive transaction or authorization information from verification evidence. Zero-knowledge proofs and cryptographic commitments enable policy and redaction verification without revealing unnecessary sensitive information. Because transaction content is stored in the ledger, the consortium nodes that maintain the PBN can access it; the content-privacy goal therefore applies to auditors and external parties, whereas authorization credentials and attributes remain hidden from all participants through PQZK. Protecting transaction content from the storing nodes themselves, e.g., through on-chain encryption, is outside the scope of this work.

6) Quantum-Capable Adversary: The adversary may possess quantum-computing capabilities. VeRedact-PQ therefore relies on post-quantum chameleon hashing and post-quantum digital signatures for security-critical operations.

We assume that the adversary cannot break the underlying post-quantum cryptographic assumptions, forge valid signatures, find hash collisions, or generate a valid zero-knowledge proof for a false statement except with negligible probability. Compromise of the committee threshold, violation of the blockchain consensus assumption, denial-of-service attacks, endpoint compromise, and implementation-level side-channel attacks are outside the scope of this work.

C. System Process

The VeRedact-PQ framework operates through 6 sequential phases where its details are described below. Table II summarizes the major notations used throughout the framework.

Phase 1: System Initialization: This phase initializes the cryptographic parameters, registered entities, redaction policies, and distributed redaction authority required by VeRedact-PQ. Let λ denote the security parameter and e denote the current authorization epoch.

Step 1: Global Parameter Setup. Given the security parameter 1^λ, the consortium initializes the cryptographic primitives and generates the global public parameters

$$PP = (\lambda, H, H_1, H_2, H_A, \mathsf{PQCH}, \mathsf{PQSIG}, \mathsf{PQZK}, \mathsf{Merkle}, \mathsf{PRF}) \tag{1}$$

where H is a post-quantum-secure hash function, PQCH denotes the post-quantum chameleon-hash scheme, PQSIG denotes the post-quantum digital-signature scheme, PQZK denotes the post-quantum zero-knowledge proof system, and Merkle denotes the authenticated Merkle-tree construction. In addition, H_1, H_2, and H_A are domain-separated instances of H, and PRF denotes a quantum-secure pseudorandom function. The public parameters PP are made available to all registered participants.

Step 2: Entity Registration and Key Generation. Each participating entity U_i, including data owners, authorized requesters, committee members, the Validation and Policy Service (VPS), the PBN checkpoint authority, the PBN audit service, and auditors, registers with the permissioned blockchain and generates a post-quantum signature key pair

$$(pk_i, sk_i) \leftarrow \mathsf{PQSIG.KeyGen}(1^\lambda) \tag{2}$$

The tuple $(ID_i, Role_i, pk_i)$ is recorded in the consortium membership registry, while sk_i is retained privately by U_i. In particular, the key pair of the VPS is denoted by (pk_V, sk_V); sk_V is used to issue validation attestations in Phase 3, and pk_V is used to verify them in Phases 4 and 6. Likewise, the checkpoint authority, operated by the PBN ordering service, holds $(pk_{\mathrm{CP}}, sk_{\mathrm{CP}})$ for signing batch checkpoints, and the audit service holds (pk_A, sk_A) for signing audit responses. A committee member C_k acting in epoch e signs with its registered key pair, denoted $(pk_{e,k}, sk_{e,k})$. The consortium further samples the PRF keys

$$K_{\mathrm{idx}} \xleftarrow{\$} \{0,1\}^\lambda, \qquad K_A \xleftarrow{\$} \{0,1\}^\lambda \tag{3}$$

where λ is chosen for the target post-quantum security level ; for example, $\lambda = 256$ provides 128-bit security against quantum key search . K_{idx} derives SA-RLI lookup tokens and is shared by the PBN and the VPS, while K_A derives

RAI audit tokens and is held by the PBN audit service.

Step 3: Redaction Policy Registration. For each supported transaction or data class, the consortium defines a redaction policy P_j specifying the authorized requester roles, permitted redaction operations, applicable conditions, and validity period. The policy state is committed as

$$C_{P_j} = H(PID_j \parallel P_j \parallel v_j \parallel e_P) \tag{4}$$

where PID_j and v_j denote the policy identifier and version, respectively , and e_P is the policy-commitment epoch. The tuple $(PID_j, C_{P_j}, v_j, e_P)$ is registered on-chain to provide an authenticated reference for subsequent policy verification.

Step 4: Epoch Committee Formation. At the beginning of epoch e, the consortium establishes an n-member redaction committee

$$\mathcal{C}_e = (C_1, C_2, \ldots, C_n), \qquad t \le n \tag{5}$$

where at least t committee members are required to exercise the redaction capability. The committee configuration $(e, \mathcal{C}_e, t)$ is recorded on-chain, binding subsequent redaction authorization to the committee active during epoch e.

Step 5: Distributed PQCH Trapdoor Setup. At system initialization, the members of the initial committee $\mathcal{C}_{e_0}$ jointly execute the distributed key generation algorithm of the PQCH scheme:

$$(pk_{\mathrm{CH}}, \{td_{e_0,k}\}_{k=1}^{n}) \leftarrow \mathsf{PQCH.DKeyGen}(PP, \mathcal{C}_{e_0}, t) \tag{6}$$

where pk_{CH} is the public chameleon-hash key and $td_{e,k}$ denotes the trapdoor share held by committee member C_k. The key pk_{CH} is generated once, recorded on the PBN, and remains fixed for the lifetime of the system. At each epoch transition $e-1 \to e$, the outgoing committee proactively reshares the trapdoor to the incoming committee:

$$\{td_{e,k}\}_{k=1}^{n} \leftarrow \mathsf{PQCH.Reshare}\big(\{td_{e-1,k}\}, \mathcal{C}_e, t\big) \tag{7}$$

after which previous-epoch shares are erased. Consequently, every batch commitment, regardless of the epoch in which it was created, can be adapted by the currently active committee. No individual committee member possesses the complete redaction trapdoor, and at least t authorized members are required to exercise the distributed redaction capability.

Step 6: Post-Quantum ZK Parameter Setup. The consortium initializes a post-quantum zero-knowledge proof (PQZK) system for privacy-preserving policy-compliance verification. For the policy relation $\mathcal{R}_P$, the system generates

$$(pp_{\mathrm{PQZK}}, pk_{\mathrm{PQZK}}, vk_{\mathrm{PQZK}}) \leftarrow \mathsf{PQZK.Setup}(1^\lambda, \mathcal{R}_P) \tag{8}$$

where pp_{PQZK} denotes the public proof parameters, pk_{PQZK} and vk_{PQZK} denote the proving and verification parameters, respectively. The PQZK construction is assumed to provide completeness, soundness, and zero-knowledge security against quantum-capable adversaries. The PQZK instantiation is assumed to employ a transparent (trapdoor-free) setup, and the resulting public and verification parameters are published on the PBN, whereas any prover-specific secret state, if required by the selected instantiation, is securely maintained by the corresponding prover. A transparent setup is adopted so that PQZK soundness does not depend on any party erasing setup secrets, and the parameters need not be regenerated when new policy types are registered. All records registered in this phase, including policy commitments, committee configurations, and public keys, are submitted as transactions signed with the registering consortium members' post-quantum keys, so that their authenticity does not rely solely on classical PBN credentials.

Phase 2: Redaction-Aware Transaction Commitment: This phase authenticates submitted transactions and establishes immutable provenance, redaction-aware commitments, and an authenticated lookup index for subsequent authorization, redaction, and auditing. Unless stated otherwise, the operations in this phase are performed by the PBN.

Step 1: Transaction Submission and Authentication. A registered data owner DO_i prepares transaction m_i with identifier TID_i, data type DT_i, redaction policy PID_i, and timestamp ts_i. The data owner first computes the salted content commitment

$$D_i = H(m_i \parallel \rho_i), \qquad \rho_i \xleftarrow{\$} \{0,1\}^\lambda \tag{9}$$

which hides low-entropy content, and signs the commitment rather than the raw content:

$$\sigma_i \leftarrow \mathsf{PQSIG.Sign}\big(sk_{DO_i}, H(TID_i \parallel D_i \parallel DT_i \parallel PID_i \parallel ts_i)\big) \tag{10}$$

The submitted transaction is

$$TX_i = (TID_i, m_i, \rho_i, DT_i, PID_i, ts_i, \sigma_i) \tag{11}$$

The permissioned blockchain recomputes D_i and verifies the registered signing key, signature, transaction uniqueness, and policy reference before accepting TX_i. The verified key determines the data-owner identity ID_{DO_i}.

To preserve original transaction provenance across subsequent redactions, the system constructs

$$C_i^{\mathrm{orig}} = H(TID_i \parallel D_i \parallel \sigma_i) \tag{12}$$

The commitment is anchored in a non-redactable provenance record containing $(TID_i, DT_i, PID_i, ts_i, D_i, \sigma_i)$. Because σ_i covers the salted commitment D_i rather than m_i, the original signature remains verifiable after m_i is redacted, and the original content need not be retained. Thus, subsequent redactions do not require the data owner to re-sign modified content.

Step 2: Redaction-Aware Commitment Construction. For each accepted transaction, the system generates a fresh nonce n_i and derives

$$Tag_i = H(TID_i \parallel n_i \parallel ts_i) \tag{13}$$

The Redaction-Aware Dual Commitment (RADC) separates immutable transaction-policy information from redactable content:

$$I_i = H(TID_i \parallel ID_{DO_i} \parallel DT_i \parallel PID_i \parallel ts_i \parallel Tag_i) \tag{14}$$

and the redactable component is the salted content commitment D_i from Step 1. The content m_i and its randomness ρ_i

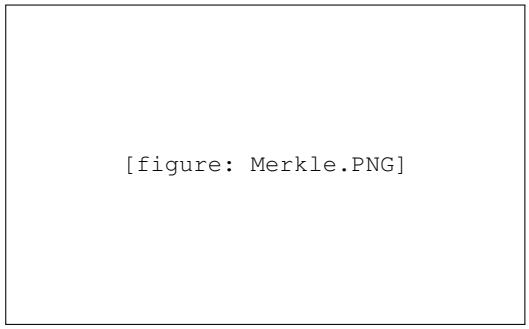


Fig. 2: Structure of the Redaction Merkle Tree and PQCH Batch Commitment

are stored in the transaction body on the PBN; an authorized redaction replaces them in place with (m'_i, ρ'_i). The initial authenticated transaction leaf is

$$L_i = H(I_i \parallel D_i) \tag{15}$$

An authorized redaction changes D_i while preserving I_i. The original signature and provenance commitment remain unchanged and are verified independently of the current redactable content state.

Step 3: Merkle and PQCH Batch Commitment. Authenticated transaction leaves are organized into batch $\mathcal{T}_b = (L_1, \ldots, L_N)$ and committed through

$$MR_b = \mathsf{Merkle.Root}(L_1, \ldots, L_N) \tag{16}$$

The system then generates the post-quantum chameleon-hash commitment

$$CH_b = \mathsf{PQCH.Hash}(pk_{\mathrm{CH}}, MR_b, r_b) \tag{17}$$

where r_b is fresh randomness , recorded in the batch checkpoint so that later adaptations can be verified. The initial batch version is $v_b = 0$.

The Merkle structure retains the authentication information needed for incremental updates. Multiple authorized redactions targeting the same batch can therefore share authentication paths and produce a single updated batch root. Because pk_{CH} is fixed and its trapdoor is reshared across epochs, the batch commitment can be adapted by whichever committee is active when a redaction is authorized.

Step 4: State Anchoring and Scalable Redaction Lookup Index. VeRedact-PQ constructs a *Sharded Authenticated Redaction Lookup Index (SA-RLI)* for efficient transaction resolution. For each transaction, the system derives

$$\tau_i = \mathsf{PRF}_{K_{\mathrm{idx}}}(TID_i) \tag{18}$$

and constructs the authenticated entry

$$E_i = \big(\tau_i, Tag_i, b, pos_i, PID_i, e_i, I_i, D_i, ptr_i\big) \tag{19}$$

where e_i denotes the epoch of the indexed transaction state and ptr_i references the transaction leaf and its Merkle authentication evidence. The current batch version v_b is obtained from the authenticated batch checkpoint rather than duplicated in each entry.

Each entry is assigned to an SA-RLI shard and logical lookup bucket using

$$sid_i = H_1(\tau_i) \bmod S \tag{20}$$

$$bid_i = H_2(\tau_i) \bmod B \tag{21}$$

where S and B denote the numbers of index shards and logical lookup buckets, respectively.

Entries within each shard are canonically ordered by their logical bucket and lookup token. Each shard maintains an authenticated root

$$R_s = \mathsf{Merkle.Root}(E_{s,1}, \ldots, E_{s,n_s}) \tag{22}$$

and the shard roots are combined into the global index root

$$R_{\mathrm{RLI}} = \mathsf{Merkle.Root}(R_1, \ldots, R_S) \tag{23}$$

After finalizing the batch and its corresponding index updates, the system assigns a monotonically increasing index snapshot identifier v_{RLI} and constructs

$$\begin{aligned} A_b = \big(&b, CH_b, MR_b, r_b, R_{\mathrm{RLI}}, \\ &v_{\mathrm{RLI}}, e, v_b, ts_b, \sigma_b\big) \end{aligned} \tag{24}$$

where e is the checkpoint-finalization epoch. The registered checkpoint authority , operated by the PBN ordering service, signs

$$\begin{aligned} \sigma_b = \mathsf{PQSIG.Sign}\big(&sk_{\mathrm{CP}}, H(b \parallel CH_b \parallel MR_b \parallel r_b \\ &\parallel R_{\mathrm{RLI}} \parallel v_{\mathrm{RLI}} \\ &\parallel e \parallel v_b \parallel ts_b)\big) \end{aligned} \tag{25}$$

The checkpoint is anchored to the permissioned blockchain, binding the batch commitment and version to a finalized SA-RLI snapshot. Index snapshots are finalized consistently with the corresponding batch-state updates.

Each shard maintains a Cuckoo filter CF_s over its indexed lookup tokens. The filter screens nonexistent candidates, whereas positive results require authentication against the finalized shard and global index roots. A negative result is authoritative only when the filter is synchronized with the selected finalized index snapshot; otherwise, the system performs authenticated index lookup.

SA-RLI thereby resolves a redaction target to its authenticated transaction batch, leaf position, policy, and current RADC state without scanning blockchain transactions. Subsequent content modifications update only the affected entries and their index authentication paths, while batch-version changes are recorded in the corresponding authenticated checkpoint.

Phase 3: Redaction Request Authentication and PQZK-Based Policy Verification: This phase validates redaction requests through staged admission, authenticated transaction resolution, post-quantum requester authentication, and public and private policy verification. Accepted requests receive PQ-signed validation attestations before ABRRR authorization.

Step 1: Request Admission and Authenticated Transaction Resolution. A requester U_r prepares

$$R_i = (ID_r, TID_i, op_i, D'_i, e, ts_r, n_r), \tag{26}$$

where op_i is the requested operation, e is the authorization epoch, and (ts_r, n_r) provides request freshness. The requester also prepares the redaction payload

$$X_i^{\mathrm{red}} = (m_i', \rho_i'), \qquad D_i' = H(m_i' \parallel \rho_i'), \tag{27}$$

where m_i' is the proposed redacted content and ρ_i' is fresh randomness. Thus, R_i binds the requested redaction to m_i' through D_i' without including the redacted content directly. The payload X_i^{red} is retained for execution in Phase 5, where its opening against D_i' is verified before the state update. VPS first screens the request format, timestamp, nonce, and epoch.

For an admissible request, VPS derives

$$\tau_i = \mathsf{PRF}_{K_{\mathrm{idx}}}(TID_i) \tag{28}$$

and determines the SA-RLI shard and bucket:

$$sid_i = H_1(\tau_i) \bmod S, \qquad bid_i = H_2(\tau_i) \bmod B \tag{29}$$

The synchronized Cuckoo filter CF_{sid_i} screens absent tokens. For a positive result, or when filter synchronization is uncertain, VPS performs authenticated lookup and retrieves

$$E_i = \big(\tau_i, Tag_i, b, pos_i, PID_i, e_i, I_i, D_i, ptr_i\big) \tag{30}$$

VPS verifies entry membership against the shard root R_{sid_i} and the global root R_{RLI} of the latest finalized index snapshot v_{RLI}. It then retrieves the authenticated checkpoint A_b for batch b, obtains its current version v_b, and verifies the transaction leaf

$$L_i = H(I_i \parallel D_i) \tag{31}$$

at position pos_i against the checkpoint's batch root MR_b. The index entry and batch checkpoint must represent the same finalized transaction state, while the checkpoint and index snapshot must be consistent with the current blockchain-anchored state.

The indexed epoch e_i is retained as transaction-state metadata and need not equal the requested authorization epoch e. The resolved transaction state and policy reference are retained by VPS and disclosed to U_r only after the requester authentication in Step 2 succeeds. They are then used to construct the policy-compliance proof in the subsequent steps. **Step 2: Post-Quantum Requester Authentication.** The requester signs the complete request:

$$\sigma_i^R \leftarrow \mathsf{PQSIG.Sign}(sk_r, H(R_i)) \tag{32}$$

and submits (R_i, σ_i^R). Using the registered public key pk_r, VPS verifies

$$\mathsf{PQSIG.Verify}(pk_r, H(R_i), \sigma_i^R) = 1 \tag{33}$$

It also checks the binding between ID_r and pk_r, requester registration, and eligibility in epoch e. A nonce is consumed within the authenticated requester's replay domain only after successful authentication.

Step 3: Public Policy and State Validation. Using the authenticated PID_i, VPS retrieves the applicable policy P_i, version v_i, and on-chain commitment C_{P_i}, verifying

$$C_{P_i} \stackrel{?}{=} H(PID_i \parallel P_i \parallel v_i \parallel e_P) \tag{34}$$

where e_P is the policy-commitment epoch.

VPS applies the registered policy-version and epoch-transition rules to establish that P_i governs the target transaction and is valid for authorization epoch e. It checks the permitted operation, public policy conditions, and freshness of the authenticated transaction state (I_i, D_i, v_b, e_i).

The requester obtains the authenticated policy commitment and verified state needed to construct the PQZK statement. Requests failing these public checks are rejected before PQZK verification.

Step 4: PQZK-Based Private Policy Verification. The requester proves satisfaction of the private conditions of P_i without disclosing its credentials or sensitive authorization attributes. The public statement is

$$x_i = \big(ID_r, TID_i, I_i, D_i, D_i', C_{P_i}, op_i, b, v_b, e_i, e, ts_r, n_r\big) \tag{35}$$

The witness w_i contains the private credentials, authorization attributes, and sensitive policy evidence.

The policy relation requires

$$\begin{aligned} \mathcal{R}_P(x_i, w_i) = 1 \iff\ & \mathsf{ValidCred}(w_i, C_{P_i}) \\ & \wedge \mathsf{RequesterBound}(w_i, ID_r) \\ & \wedge \mathsf{PrivatePolicy}(x_i, w_i) \end{aligned} \tag{36}$$

where credential validity includes the applicable expiry and revocation conditions. Requester binding establishes credential ownership or valid delegation rather than relying on an unverified identity claim.

The requester generates

$$\pi_i^{PQ} \leftarrow \mathsf{PQZK.Prove}(pk_{\mathrm{PQZK}}, x_i, w_i) \tag{37}$$

and VPS accepts only if

$$\mathsf{PQZK.Verify}(vk_{\mathrm{PQZK}}, x_i, \pi_i^{PQ}) = 1 \tag{38}$$

The public statement binds the proof to the authenticated requester, request, policy commitment, and verified transaction state, preventing substitution across requests or state versions.

Step 5: Validated Request Commitment and Attestation. For each accepted request, VPS assigns a unique identifier RID_i and constructs

$$\begin{aligned} C_i^{VR} = H\big(&RID_i \parallel H(R_i) \parallel I_i \parallel D_i \\ &\parallel C_{P_i} \parallel b \parallel pos_i \parallel v_b \parallel e_i \parallel H(\pi_i^{PQ})\big) \end{aligned} \tag{39}$$

VPS then signs the validated-request commitment under authorization epoch e:

$$\alpha_i = \mathsf{PQSIG.Sign}\left(sk_V, H(RID_i \parallel C_i^{VR} \parallel e)\right) \tag{40}$$

The compact validated request is

$$\begin{aligned} VR_i = \big(&RID_i, C_i^{VR}, b, pos_i, PID_i, \\ &v_b, e_i, e, \sigma_i^R, H(\pi_i^{PQ}), \alpha_i\big) \end{aligned} \tag{41}$$

The complete request R_i, public statement x_i, proof π_i^{PQ}, and authenticated policy and transaction evidence remain available for independent verification and challenged audits.

Validated requests enter the pending queue $\mathcal{Q}_e$. Upon admitting VR_i to $\mathcal{Q}_e$, the VPS returns to the requester a signed

admission receipt

$$rc_i = \mathsf{PQSIG.Sign}\big(sk_V, H(RID_i \parallel H(R_i) \parallel ts_i^{rc})\big) \quad (42)$$

where ts_i^{rc} is the admission time. If the request is neither included in an authorized batch nor explicitly rejected within $T_{\max}$ of ts_i^{rc}, the requester may present rc_i to the committee as verifiable evidence that the VPS admitted but did not process the request. Phase 4 verifies their attestations and current-state freshness before applying ABRRR multi-party authorization.

Phase 4: Adaptive Batch Formation and Multi-Party Committee Authorization: This phase employs Adaptive Batch Round Redaction Request (ABRRR) to group validated requests according to workload and waiting-time constraints. The active epoch committee verifies their attestations and freshness before issuing t-of-n authorization over an authenticated batch commitment.

Step 1: Adaptive Batch Formation. The VPS maintains the pending queue and executes ABRRR. Let $\mathcal{Q}_e$ be the queue of validated requests for authorization epoch e, λ_e the observed arrival rate, and $T_{\max}$ the maximum waiting time. ABRRR determines the target batch size as

$$\widehat{B}_e = f(\lambda_e, |\mathcal{Q}_e|, T_{\max}) \quad (43)$$

$$B_e^* = \min\{B_{\max}, \max\{B_{\min}, \lceil \widehat{B}_e \rceil\}\} \quad (44)$$

where $B_{\min}$ and $B_{\max}$ are the permitted batch-size bounds. A batch closes when B_e^* requests are available or its oldest request reaches $T_{\max}$. The resulting candidate batch is

$$\mathcal{B}_e = (VR_1, \ldots, VR_m), \qquad 1 \le m \le B_e^* \quad (45)$$

Step 2: Attestation and State-Freshness Verification. For each $VR_i \in \mathcal{B}_e$, the committee verifies the Phase 3 attestation using the registered VPS public key:

$$\mathsf{PQSIG.Verify}\big(pk_V, H(RID_i \parallel C_i^{VR} \parallel e), \alpha_i\big) = 1 \quad (46)$$

It also checks that the request's supporting evidence reconstructs C_i^{VR} and that the authenticated requester, policy commitment, and PQZK-proof hash match the attested request. Routine authorization relies on the VPS attestation rather than repeating PQZK verification.

The committee then retrieves the latest finalized transaction state and checks

$$(D_i, v_b, e_i) \stackrel{?}{=} (D_i^{cur}, v_b^{cur}, e_i^{cur}) \quad (47)$$

where e_i^{cur} is the current indexed transaction-state epoch, not the current authorization epoch. The committee separately verifies that epoch e remains active and that the attested policy version is still applicable. Requests with invalid attestations, stale states, or expired authorization are excluded and returned for revalidation.

Step 3: PQZK-Bound Batch Commitment. For each eligible request, the system constructs

$$\eta_i = H\big(RID_i \parallel C_i^{VR} \parallel H(\pi_i^{PQ}) \parallel \alpha_i \parallel b \parallel v_b \parallel e_i \parallel e\big) \quad (48)$$

The eligible request leaves are canonically ordered and aggregated into

$$R_e^{VR} = \mathsf{Merkle.Root}(\eta_1, \ldots, \eta_m) \quad (49)$$

This root commits to the independently verified PQZK evidence and validation attestations without aggregating the underlying PQZK proofs.

The ABRRR batch commitment is

$$C_e^B = H(BID_e \parallel e \parallel m \parallel R_e^{VR} \parallel ts_e) \quad (50)$$

where BID_e uniquely identifies the authorization batch and ts_e is its formation timestamp. Here, m denotes the number of requests remaining after Step 2; the batch commitment is finalized only after ineligible requests have been removed.

Step 4: Multi-Party Committee Authorization. Each approving member C_k of the active committee $\mathcal{C}_e$ independently verifies the finalized batch commitment and signs

$$\sigma_{e,k}^B \leftarrow \mathsf{PQSIG.Sign}\left(sk_{e,k}, H(C_e^B \parallel e)\right) \quad (51)$$

The set of valid approvals is

$$\mathcal{A}_e = \big\{(k, \sigma_{e,k}^B) : C_k \in \mathcal{C}_e, \; \mathsf{PQSIG.Verify}\big(pk_{e,k}, H(C_e^B \parallel e), \sigma_{e,k}^B\big) = 1\big\} \quad (52)$$

The batch is authorized only if

$$|\mathcal{A}_e| \ge t \quad (53)$$

with approvals from distinct registered committee members. Its authorization evidence is

$$Auth_e^B = \big(BID_e, C_e^B, R_e^{VR}, \mathcal{A}_e, e, ts_e\big) \quad (54)$$

This construction uses t independent post-quantum signatures and does not assume native threshold-signature support.

Step 5: Policy-Bound Authorization Evidence. For each authorized request, the system generates a Merkle proof MP_i^B of η_i under R_e^{VR}. The authorization component of the Policy-Bound Batch Redaction Proof (PBRP) is

$$PBRP_i^{auth} = \big(RID_i, \eta_i, MP_i^B, R_e^{VR}, C_e^B, Auth_e^B\big) \quad (55)$$

This evidence binds each validated request to the committee-authorized batch while allowing shared authorization evidence to be verified once per batch.

Only batches satisfying the threshold authorization condition proceed to Phase 5. Before execution, Phase 5 rechecks transaction-state and policy freshness, then binds the executed RADC transition, BIMC update, and PQCH adaptation to $PBRP_i^{auth}$ to construct the final PBRP.

Phase 5: Authorized Batch Redaction and Verifiable State Update: This phase executes the ABRRR-authorized requests using coalesced Merkle updates and distributed PQCH adaptation. For each affected transaction batch, VeRedact-PQ produces one authenticated state transition and binds its execution evidence to the Phase 4 authorization.

Step 1: Request Grouping and Execution-State Validation. Given an authorized batch $\mathcal{B}_e$, the system verifies $Auth_e^B$ and

each $PBRP_i^{auth}$, then partitions the requests:

$$\mathcal{B}_e = \bigcup_{b \in \Omega_e} \mathcal{B}_{e,b} \tag{56}$$

$$\mathcal{B}_{e,b} = \{VR_i \in \mathcal{B}_e : \mathsf{batch}(VR_i) = b\} \tag{57}$$

where Ω_e denotes the affected transaction batches.

Before execution, each request must satisfy

$$(D_i, v_b, e_i) \stackrel{?}{=} (D_i^{cur}, v_b^{cur}, e_i^{cur}) \tag{58}$$

The system also verifies that the Phase 4 authorization remains valid in epoch e, the applicable policy is current, and the approved operation and proposed commitment match the attested request.

Requests targeting the same transaction are checked for conflicts. At most one resulting content commitment is selected per transaction within an execution round; conflicting requests are deferred for separate authorization or revalidation. Stale or otherwise ineligible requests are excluded. The remaining requests form the execution set $\mathcal{B}_{e,b}^* \subseteq \mathcal{B}_{e,b}$.

Step 2: RADC State Transition. For each $VR_i \in B_{e,b}^*$, the system retrieves the redaction payload

$$X_i^{\mathrm{red}} = (m_i', \rho_i') \tag{59}$$

submitted with the corresponding request and verifies that it opens the committee-authorized content commitment:

$$H(m_i' \parallel \rho_i') \stackrel{?}{=} D_i' \tag{60}$$

Only a payload satisfying this binding is executed. The Redaction-Aware Dual Commitment (RADC) transition is then

$$D_i' = H(m_i' \parallel \rho_i'), \qquad I_i' = I_i \tag{61}$$

with updated authenticated leaf

$$L_i' = H(I_i \parallel D_i') \tag{62}$$

The original data-owner signature and authenticated provenance commitment remain unchanged. The executed operation must also satisfy the scope of the committee-authorized request. **Step 3: Coalesced BIMC and Distributed PQCH Adaptation.** For each nonempty execution set $\mathcal{B}_{e,b}^*$, let $\mathcal{L}_b$ and $\mathcal{L}_b'$ be the corresponding old and new leaves at their authenticated positions. BIMC verifies their compact multiproof MP_b^{multi} and computes

$$MR_b' = \mathsf{BIMC.Update}(MR_b, \mathcal{L}_b, \mathcal{L}_b', MP_b^{multi}) \tag{63}$$

Shared authentication paths are processed once for all modifications within batch b.

Using the PQCH key associated with the original batch commitment, each participating committee member C_k computes

$$\delta_{b,k} \leftarrow \mathsf{PQCH.PartAdapt}(td_{e,k}, MR_b, r_b, MR_b', BID_e) \tag{64}$$

Only shares generated under a valid trapdoor-sharing configuration for pk_{CH} are accepted. After verifying t distinct shares, the system combines them:

$$r_b' \leftarrow \mathsf{PQCH.Combine}(\{\delta_{b,k}\}_{k \in \mathcal{T}_b}) \tag{65}$$

$$|\mathcal{T}_b| \geq t \tag{66}$$

The resulting adaptation must satisfy

$$\begin{aligned} &\mathsf{PQCH.Hash}(pk_{\mathrm{CH}}, MR_b, r_b) \\ &\quad = \mathsf{PQCH.Hash}(pk_{\mathrm{CH}}, MR_b', r_b') = CH_b \end{aligned} \tag{67}$$

Thus, all modifications within one transaction batch share one BIMC root transition and one distributed PQCH adaptation. Distinct batches may be prepared in parallel.

Step 4: Atomic Authenticated State Finalization. After successful adaptation, each affected batch advances its version:

$$v_b' = v_b + 1 \tag{68}$$

For every executed transaction, its SA-RLI entry is updated as

$$E_i' = \big(\tau_i, Tag_i, b, pos_i, PID_i, e_i', I_i, D_i', ptr_i'\big) \tag{69}$$

where $e_i' = e$ identifies the epoch of the executed transaction-state update. The reference ptr_i' resolves current authentication evidence under MR_b'. Unmodified entries retain their content commitments and state epochs, while their Merkle evidence is refreshed or generated against the current batch root when queried.

Let Ω_S denote the index shards containing updated entries. Their roots are recomputed as

$$R_s^* = \begin{cases} R_s', & s \in \Omega_S \\ R_s, & s \notin \Omega_S \end{cases} \tag{70}$$

yielding the new global index root

$$R_{\mathrm{RLI}}' = \mathsf{Merkle.Root}(R_1^*, \ldots, R_S^*) \tag{71}$$

The system advances the index snapshot identifier:

$$v_{\mathrm{RLI}}' = v_{\mathrm{RLI}} + 1 \tag{72}$$

For each affected batch, the updated checkpoint is

$$A_b' = \big(b, CH_b, MR_b', r_b', R_{\mathrm{RLI}}', v_{\mathrm{RLI}}', e, v_b', ts_b', \sigma_b'\big) \tag{73}$$

where

$$\begin{aligned} \sigma_b' = \mathsf{PQSIG.Sign}\big(sk_{\mathrm{CP}}, H(b \parallel CH_b \parallel MR_b' \parallel r_b' \\ \parallel R_{\mathrm{RLI}}' \parallel v_{\mathrm{RLI}}' \\ \parallel e \parallel v_b' \parallel ts_b')\big) \end{aligned} \tag{74}$$

All affected batch checkpoints are finalized against the same global index snapshot. The blockchain atomically commits the corresponding batch-state and SA-RLI updates after verifying the authorized transitions. If finalization fails, the prepared updates are not published as finalized state.

Step 5: PBRP and Redaction Record Generation. For each executed request, VeRedact-PQ constructs the final Policy-Bound Batch Redaction Proof:

$$\begin{aligned} PBRP_i = \big(&PBRP_i^{auth}, I_i, D_i, D_i', b, pos_i, v_b, v_b', \\ &MR_b, MR_b', CH_b, e, BID_e, \Pi_i^{exec}\big) \end{aligned} \tag{75}$$

where Π_i^{exec} contains the authenticated old/new leaf evidence,

the verified PQCH adaptation evidence, and the corresponding finalized checkpoint references. Shared batch-level evidence may be referenced rather than duplicated in every PBRP.

The compact redaction record is

$$RR_i = \big(RID_i, TID_i, BID_e, PID_i, D_i, D_i', v_b, v_b', H(PBRP_i), ts_i^{red}\big) \tag{76}$$

Only successfully executed requests produce redaction records. The complete PBRP and its supporting evidence remain available for Phase 6 auditing.

The resulting PBRP links each validated request and committee authorization to its executed RADC transition, coalesced Merkle update, PQCH adaptation, and blockchain-finalized state.

Algorithm 1 summarizes the execution. It exposes the main optimization of VeRedact-PQ: authorization is performed once per ABRRR batch, common Merkle paths are coalesced by BIMC, and PQCH adaptation is performed once per affected transaction batch with parallel threshold-share generation.

The resulting PBRP provides a verifiable link from the individually validated request and Phase 4 committee authorization to the executed RADC transition and the coalesced BIMC/PQCH state update. The redaction records RR_i are subsequently indexed by RAI in Phase 6 for privacy-preserving query-based auditing.

Phase 6: Post-Quantum Privacy-Preserving Query-Based Redaction Auditing: This phase supports authenticated, query-scoped verification of completed redactions without requiring access to sensitive transaction content. VeRedact-PQ combines a sharded Redaction Audit Index (RAI), shared batch evidence, and compact Merkle multiproofs. Routine audits verify PQ-signed validation attestations, while challenged audits add independent PQZK verification.

Step 1: Authenticated Redaction Audit Index Construction. For each finalized redaction record RR_i, the system derives

$$\tau_i^A = \mathsf{PRF}_{K_A}(TID_i) \tag{77}$$

and constructs

$$E_i^A = \big(\tau_i^A, RID_i, BID_e, PID_i, e, v_b, v_b', H(PBRP_i), ts_i^{red}, ptr_i^A\big) \tag{78}$$

where ptr_i^A references the complete PBRP and its supporting evidence. Entries are retained for historical verification rather than overwritten by later redactions.

RAI partitions the entries into S_A authenticated shards:

$$sid_i^A = H_A(\tau_i^A) \bmod S_A \tag{79}$$

$$R_{\mathrm{RAI}} = \mathsf{Merkle.Root}(R_1^A, \ldots, R_{S_A}^A) \tag{80}$$

Each shard maintains a deterministic authenticated ordering of its entries. A monotonically increasing snapshot version v_A identifies each finalized RAI state.

At the end of each audit epoch, the system constructs

$$CP_e^A = H(e \parallel R_{\mathrm{RAI}} \parallel v_A \parallel ts_e) \tag{81}$$

and anchors the checkpoint to the permissioned blockchain. Auditors use the finalized checkpoint corresponding to the requested audit snapshot.

Algorithm 1 Authorized Batch Redaction and Verifiable State Update

Require: Authorized batch $\mathcal{B}_e$, $Auth_e^B$, finalized batch/index states, threshold t
Ensure: Finalized checkpoints $\{A_b'\}$, redaction records $\{RR_i\}$

Verify $Auth_e^B$ and request membership evidence
Partition requests into $\{\mathcal{B}_{e,b}\}_{b\in\Omega_e}$
$\mathcal{P} \leftarrow \emptyset$
for all $b \in \Omega_e$ **in parallel do**
  $\mathcal{B}_{e,b}^* \leftarrow \mathsf{ValidateAndResolve}(\mathcal{B}_{e,b})$
  Exclude stale, invalid, or conflicting requests
  if $\mathcal{B}_{e,b}^* = \emptyset$ **then**
    continue
  end if
  Construct $\mathcal{L}_b$ and $\mathcal{L}_b'$ using $L_i' = H(I_i \parallel D_i')$
  Verify MP_b^{multi}
  $MR_b' \leftarrow \mathsf{BIMC.Update}(MR_b, \mathcal{L}_b, \mathcal{L}_b', MP_b^{multi})$
  for all $C_k \in \mathcal{C}_e$ **in parallel do**
    $\delta_{b,k} \leftarrow \mathsf{PQCH.PartAdapt}(td_{e,k}, MR_b, r_b, MR_b', BID_e)$
  end for
  Collect t distinct valid shares $\{\delta_{b,k}\}_{k\in\mathcal{T}_b}$
  $r_b' \leftarrow \mathsf{PQCH.Combine}(\{\delta_{b,k}\}_{k\in\mathcal{T}_b})$
  if $\mathsf{PQCH.Hash}(pk_{\mathrm{CH}}, MR_b', r_b') \neq CH_b$ **then**
    Abort preparation for batch b
  else
    $v_b' \leftarrow v_b + 1$
    Add prepared transition $(b, MR_b', r_b', v_b', \mathcal{B}_{e,b}^*)$ to $\mathcal{P}$
  end if
end for
Recheck the finalized base states for all transitions in $\mathcal{P}$
Construct updated SA-RLI entries and affected shard roots
$v_{\mathrm{RLI}}' \leftarrow v_{\mathrm{RLI}} + 1$
Compute R_{RLI}'
Construct $\{A_b'\}$ using the same finalized index snapshot
if atomic blockchain finalization succeeds **then**
  for all executed VR_i in $\mathcal{P}$ **do**
    Construct $PBRP_i$ and RR_i
  end for
  return $\{A_b'\}, \{RR_i\}$
else
  Discard prepared state updates
  return failure
end if

Step 2: PQ-Authenticated Audit Query and Resolution. An authorized auditor AU_j submits

$$Q_j^A = \big(QID_j, qtype_j, \phi_j, scope_j, e_j, [t_s, t_e], n_j, ts_j\big) \tag{82}$$

where $qtype_j$ specifies the audit operation, ϕ_j its predicates, $scope_j$ its authorized scope, and e_j the requested audit epoch.

The auditor signs

$$\sigma_j^A \leftarrow \mathsf{PQSIG.Sign}(sk_{AU_j}, H(Q_j^A)) \tag{83}$$

The audit service verifies the registered auditor's signature, query freshness, and access rights before resolving

$$\mathcal{R}_{Q_j} = \{E_i^A : E_i^A \models Q_j^A\} \tag{84}$$

Resolution uses the authenticated RAI snapshot selected by the query.

For exhaustive queries, the service must also provide authenticated coverage evidence for the requested scope. Where the available index cannot prove completeness, the response is explicitly limited to verified returned records rather than claimed to be exhaustive.

Step 3: Query-Scoped Evidence Generation. For the returned entries, the service constructs a query-scoped multiproof

$$MP_{Q_j}^A \leftarrow \mathsf{Merkle.MultiProof}(\mathcal{R}_{Q_j}, R_{\mathrm{RAI}}) \tag{85}$$

including the necessary shard-level and global authentication evidence.

The returned records are grouped by their authorization-batch identifiers:

$$\Omega_{Q_j}^B = \{BID_e : E_i^A \in \mathcal{R}_{Q_j}\} \tag{86}$$

For each distinct BID_e, the shared authorization evidence $(R_e^{VR}, C_e^B, Auth_e^B)$ is included only once.

The service selects the required individual PBRP evidence:

$$\Pi_{Q_j} = \mathsf{SelectEvidence}(qtype_j, \mathcal{R}_{Q_j}, \{PBRP_i\}) \tag{87}$$

Authorization queries require request membership and committee approvals; state-transition queries additionally require the old and new RADC commitments, batch authentication evidence, PQCH adaptation evidence, and finalized checkpoint references.

Let Ans_j denote the query result and Γ_{Q_j} the authenticated coverage evidence, when completeness is required and supported. The response body is

$$\begin{aligned} Body_j^A = \big(&QID_j, H(Q_j^A), Ans_j, \mathcal{R}_{Q_j}, MP_{Q_j}^A, \\ &\Pi_{Q_j}, \Gamma_{Q_j}, R_{\mathrm{RAI}}, v_A, ts_A\big) \end{aligned} \tag{88}$$

The audit service signs its complete canonical encoding:

$$\sigma_{\mathrm{Resp}}^A \leftarrow \mathsf{PQSIG.Sign}(sk_A, H(Body_j^A)) \tag{89}$$

The authenticated response is

$$Resp_j^A = (Body_j^A, \sigma_{\mathrm{Resp}}^A) \tag{90}$$

Step 4: Cost-Aware Post-Quantum Audit Verification. The auditor verifies the response signature, query binding, and selected blockchain-anchored RAI checkpoint. It then verifies $MP_{Q_j}^A$ and, where required, Γ_{Q_j}.

For each distinct BID_e, the auditor reconstructs C_e^B and verifies the t distinct PQ committee approvals in $Auth_e^B$ against the committee registered for epoch e. Shared authorization evidence is verified once per batch.

For each returned redaction, the auditor checks the authenticated request leaf η_i and its membership proof MP_i^B under R_e^{VR}. It verifies that the associated validated-request commitment and attestation match the authorized request.

When state-transition verification is required, the auditor checks

$$I_i' = I_i, \qquad L_i = H(I_i \parallel D_i) \tag{91}$$

$$L_i' = H(I_i \parallel D_i') \tag{92}$$

The execution evidence must authenticate L_i and L_i' at the same transaction position against the respective old and new batch roots. The auditor also checks that these roots, batch versions, and the updated SA-RLI snapshot match the finalized checkpoint evidence.

PQCH commitment preservation is verified , using r_b and r_b' recorded in the checkpoints A_b and A_b', as

$$\begin{aligned} &\mathsf{PQCH.Hash}(pk_{\mathrm{CH}}, MR_b, r_b) \\ &\quad = \mathsf{PQCH.Hash}(pk_{\mathrm{CH}}, MR_b', r_b') = CH_b \end{aligned} \tag{93}$$

The verified transition must match the operation and proposed content commitment bound to the authorized request.

VeRedact-PQ supports two verification levels. During a *normal audit*, the auditor verifies the Phase 3 validation attestation:

$$\mathsf{PQSIG.Verify}\big(pk_V, H(RID_i \parallel C_i^{VR} \parallel e), \alpha_i\big) = 1 \tag{94}$$

The auditor also verifies that the authenticated request evidence contains the committed hash $H(\pi_i^{PQ})$. This path relies on the VPS's signed validation result without independently repeating PQZK verification.

During a challenged or forensic *deep audit*, the auditor retrieves the complete supporting evidence, reconstructs and authenticates the Phase 3 public statement x_i, and verifies

$$\mathsf{PQZK.Verify}(vk_{\mathrm{PQZK}}, x_i, \pi_i^{PQ}) = 1 \tag{95}$$

The auditor additionally checks that $H(\pi_i^{PQ})$ matches the hash committed in C_i^{VR}.

Step 5: Query-Specific Audit Decision. The auditor evaluates

$$\mathsf{AuditVerify}(PP, Q_j^A, Resp_j^A) \rightarrow (ans_j, \beta_j) \tag{96}$$

where $\beta_j \in \{0, 1\}$ indicates whether all verification conditions required by $qtype_j$ are satisfied.

For exhaustive queries, acceptance additionally requires authenticated coverage of the requested scope. If completeness cannot be established, the auditor may accept the authenticity of individual returned records but must not certify the result as complete.

Routine verification uses PQ signatures, shared authorization evidence, and hash-based authentication. Query-scoped multiproofs reduce redundant Merkle operations, while independent PQZK verification is reserved for deep audits.

IV. Security Analysis

We analyze VeRedact-PQ under the threat model defined in Section III-B. Let $\mathcal{A}$ be a probabilistic quantum-polynomial-time (QPT) adversary. We assume that PQSIG is existentially unforgeable against quantum chosen-message attacks, PQZK satisfies quantum soundness and zero knowledge, PQCH provides collision resistance and threshold-controlled adaptation against quantum adversaries, H is collision resistant with

parameters selected for the required post-quantum security level, PRF is quantum-secure, and the authenticated Merkle structures are binding. We further assume that fewer than t members of each redaction committee are compromised , that previous-epoch trapdoor shares are erased after resharing, and that the underlying permissioned blockchain satisfies its prescribed consensus fault-tolerance condition.

A. Post-Quantum Request Authentication and Policy Compliance

Theorem 1 (Authenticated and Policy-Compliant Admission). An adversary cannot cause an unauthorized redaction request to obtain a valid Phase 3 validation attestation except with negligible probability.

Proof: For a request R_i to reach private policy verification, it must first satisfy

$$\mathsf{PQSIG.Verify}(pk_r, H(R_i), \sigma_i^R) = 1 \tag{97}$$

Because pk_r is bound to the registered requester, an adversary without sk_r that produces a valid signature on a new R_i directly violates the assumed unforgeability of PQSIG.

Possession of a valid requester key alone is insufficient. The authenticated SA-RLI entry binds the request to its current (I_i, D_i, b, v_b, e_i) state, while the applicable policy is committed by

$$C_{P_i} = H(PID_i \parallel P_i \parallel v_i \parallel e_P) \tag{98}$$

Replacing P_i, PID_i, v_i, or e_P while preserving C_{P_i} requires a collision in H.

After the public policy conditions are checked, private eligibility requires an accepting PQZK proof

$$\mathsf{PQZK.Verify}(vk_{\mathrm{PQZK}}, x_i, \pi_i^{PQ}) = 1 \tag{99}$$

where

$$\begin{aligned} x_i = \big(&ID_r, TID_i, I_i, D_i, D_i', C_{P_i}, \\ &op_i, b, v_b, e_i, e, ts_r, n_r\big) \end{aligned} \tag{100}$$

If the requester does not possess a witness w_i satisfying $\mathcal{R}_P(x_i, w_i) = 1$, producing an accepting proof contradicts PQZK soundness. Because x_i contains ID_r, e, ts_r, and n_r, a valid proof also cannot be transferred to another requester, authorization epoch, or request instance.

Finally, the Validation and Policy Service issues

$$\alpha_i = \mathsf{PQSIG.Sign}\left(sk_V, H(RID_i \parallel C_i^{VR} \parallel e)\right) \tag{101}$$

only after all preceding checks succeed. Therefore, obtaining a valid attestation for an unauthorized request requires forging PQSIG, breaking PQZK soundness, violating an authenticated state binding, or finding a collision in H, each of which occurs only with negligible probability under the stated assumptions. Hence, an unauthorized request cannot obtain a valid Phase 3 attestation except with negligible probability.

B. Threshold Committee Authorization and Redaction Control

Theorem 2 (Threshold Redaction Security). If fewer than t members of the active committee $\mathcal{C}_e$ are compromised, they cannot independently authorize and execute a valid redaction except with negligible probability.

Proof: Phase 4 accepts an ABRRR batch only if its authorization set satisfies

$$|\mathcal{A}_e| \geq t, \qquad \mathsf{PQSIG.Verify}(pk_{e,k}, H(C_e^B \parallel e), \sigma_{e,k}^B) = 1 \tag{102}$$

for every counted committee approval. Let the adversary control $c < t$ committee members. It can generate at most c valid signatures using the corresponding compromised keys. Reaching the threshold therefore requires at least one valid signature under an uncompromised committee key. Producing such a signature contradicts PQSIG unforgeability.

Authorization alone also cannot perform the controlled ledger adaptation. Phase 5 requires at least t valid PQCH adaptation shares:

$$r_b' = \mathsf{PQCH.Combine}\left(\{\delta_{b,k}\}_{k\in\mathcal{T}_b}\right), \qquad |\mathcal{T}_b| \geq t \tag{103}$$

Since every committee member possesses only its distributed trapdoor share $td_{e,k}$, fewer than t compromised members cannot derive a valid adaptation under the threshold-security assumption of PQCH. Because previous-epoch shares are erased after proactive resharing, shares compromised in different epochs cannot be combined to reach the threshold.

Thus, an adversary controlling fewer than t members must either forge an honest committee member's PQ signature or violate the threshold security of PQCH. Both events have negligible probability. Therefore, no sub-threshold coalition can independently authorize and execute an accepted redaction.

C. Batch Integrity and Authorization Binding

Theorem 3 (Batch Non-Substitution). After an ABRRR batch has been authorized, no request can be inserted, removed, replaced, or transferred between authorized batches without detection except with negligible probability.

Proof: Each validated request contributes the authenticated leaf

$$\eta_i = H(RID_i \parallel C_i^{VR} \parallel H(\pi_i^{PQ}) \parallel \alpha_i \parallel b \parallel v_b \parallel e_i \parallel e) \tag{104}$$

and all request leaves determine

$$R_e^{VR} = \mathsf{Merkle.Root}(\eta_1, \ldots, \eta_m) \tag{105}$$

The batch commitment

$$C_e^B = H(BID_e \parallel e \parallel m \parallel R_e^{VR} \parallel ts_e) \tag{106}$$

is the object authorized by at least t committee members.

Suppose $\mathcal{A}$ replaces η_i by η_i'. Unless $\eta_i' = \eta_i$, the authenticated Merkle root changes under the binding property of the Merkle construction. The resulting $C_e^{B'}$ therefore differs from C_e^B, and the existing committee signatures fail verification. The same argument applies to insertion or removal because m and the Merkle root are both committed.

For individual verification, $PBRP_i^{auth}$ contains the Merkle membership proof connecting η_i to the authorized R_e^{VR}. Thus, presenting a request that was not in the authorized batch requires forging a valid Merkle path to the same root or finding

a collision in H. Both contradict the assumed authenticated-data- structure security. Hence, batch manipulation is detectable except with negligible probability.

D. State Freshness and Replay Resistance

Theorem 4 (State-Bound Authorization). A request or authorization generated for transaction state (D_i, v_b, e_i) cannot be validly reused after that state or authorization epoch changes.

Proof: Phase 3 binds (D_i, b, v_b, e_i, e) to the public PQZK statement, validated-request commitment, and validation attestation. Phase 4 further includes these values in η_i, which is committed by R_e^{VR} and C_e^B.

Before authorization and again immediately before execution, the system verifies

$$(D_i, v_b, e_i) \stackrel{?}{=} (D_i^{cur}, v_b^{cur}, e_i^{cur}) \tag{107}$$

After successful redaction,

$$v_b' = v_b + 1 \tag{108}$$

Therefore, previously generated evidence containing v_b fails the freshness test against v_b'. In addition, the authorization epoch e is bound into α_i, η_i, and C_e^B, and Phases 4 and 5 separately verify that e remains active; evidence authorized in an expired epoch is therefore rejected even when the transaction state is unchanged.

An adversary cannot alter the version or epoch inside existing evidence because doing so changes the associated hash commitments, PQZK statement, Merkle leaf, and committee-signed batch commitment. Successful alteration would therefore require a hash collision, signature forgery, or new valid authorization. Consequently, stale authorization and replayed requests cannot be accepted except with negligible probability.

E. Controlled Redaction and Ledger Consistency

Theorem 5 (Policy-Bound Redaction Integrity). An accepted VeRedact-PQ redaction modifies only the authorized redactable content state while preserving the immutable transaction-policy binding and PQCH commitment.

Proof: The Redaction-Aware Dual Commitment (RADC) separates the transaction state into

$$I_i = H(TID_i \parallel ID_{DO_i} \parallel DT_i \parallel PID_i \parallel ts_i \parallel Tag_i) \tag{109}$$

$$D_i = H(m_i \parallel \rho_i) \tag{110}$$

For an authorized redaction $m_i \to m_i'$, Phase 5 requires

$$I_i' = I_i \tag{111}$$

$$D_i' = H(m_i' \parallel \rho_i') \tag{112}$$

Hence, changing the transaction identity, owner, data type, policy, timestamp, or transaction tag while preserving I_i requires a collision in H.

All authorized changes affecting transaction batch b are committed to the updated Merkle root MR_b'. Ledger continuity is accepted only when

$$\begin{aligned}&\mathsf{PQCH.Hash}(pk_{\mathrm{CH}}, MR_b, r_b) = \\ &\mathsf{PQCH.Hash}(pk_{\mathrm{CH}}, MR_b', r_b') = CH_b\end{aligned} \tag{113}$$

For $MR_b' \neq MR_b$, generating valid r_b' without the required threshold adaptation violates the security of PQCH. By Theorem 2, fewer than t compromised committee members cannot generate such an adaptation.

Therefore, an accepted transition preserves I_i and CH_b while changing only the committee-authorized content state D_i. Any unauthorized modification must break either the RADC hash binding, Merkle authentication, threshold PQCH security, or committee authorization, all of which succeed only with negligible probability.

F. End-to-End Redaction Accountability

Theorem 6 (Verifiable Redaction Accountability). Every successfully executed redaction is cryptographically traceable to its independently validated request, applicable policy and state, committee-authorized ABRRR batch, and resulting ledger transition.

Proof: For every executed request, VeRedact-PQ constructs

$$\begin{aligned}PBRP_i = \big(&PBRP_i^{auth}, I_i, D_i, D_i', b, pos_i, v_b, v_b', \\ &MR_b, MR_b', CH_b, e, BID_e, \Pi_i^{exec}\big)\end{aligned} \tag{114}$$

By Theorem 1, the validated request represented inside $PBRP_i^{auth}$ has passed requester authentication and policy verification. By Theorem 3, its membership proof binds it to the specific R_e^{VR} and committee-authorized C_e^B. By Theorem 4, the authorization is bound to the applicable transaction version and epoch. Finally, by Theorem 5, (MR_b, MR_b', CH_b) binds the evidence to the controlled state transition actually executed , while Π_i^{exec} authenticates the old and new leaves at position pos_i and links them to the finalized checkpoints.

The redaction record additionally contains

$$H(PBRP_i) \tag{115}$$

which is authenticated by RAI and its blockchain-anchored checkpoint. Replacing any component of $PBRP_i$ while retaining the same digest requires a collision in H. Replacing the authenticated audit entry requires violating the Merkle binding or anchored checkpoint.

Hence, a valid redaction record establishes a continuous cryptographic chain

$$\begin{aligned}&\text{authenticated request} \Rightarrow \text{policy validation} \\ &\Rightarrow \text{batch authorization} \Rightarrow \text{authorized state transition} \\ &\Rightarrow \text{authenticated audit record.}\end{aligned} \tag{116}$$

Breaking this chain requires violating at least one of the stated cryptographic assumptions. Therefore, accepted redactions remain independently accountable except with negligible probability.

G. Privacy-Preserving and Fresh Auditing

Theorem 7 (Audit Privacy and Integrity). Under PQZK zero knowledge, PRF security, hash collision resistance, PQSIG unforgeability, and Merkle binding, an authorized auditor can verify the requested redaction properties without learning the private policy witness or requiring disclosure

of the original transaction content, while modification or substitution of returned authenticated evidence is detectable.

Proof: The private credentials and authorization attributes used in Phase 3 occur only in witness w_i. PQZK zero knowledge guarantees that π_i^{PQ} reveals no information about w_i beyond the truth of

$$\mathcal{R}_P(x_i, w_i) = 1 \tag{117}$$

Routine auditing does not require w_i and normally does not process π_i^{PQ}; instead, it verifies the PQ-signed validation attestation α_i and the committed $H(\pi_i^{PQ})$. Full proof verification is performed only when a challenged or forensic audit requires it.

RAI further replaces direct transaction lookup identifiers with

$$\tau_i^A = \mathsf{PRF}_{K_A}(TID_i) \tag{118}$$

and stores only compact metadata and $H(PBRP_i)$ in the authenticated index. The content commitments D_i and D_i' exposed in audit evidence are salted with ρ_i and ρ_i', so they reveal no information about low-entropy content under the hiding property of the commitment. Under PRF security, a party without K_A cannot invert τ_i^A to recover TID_i better than allowed by the underlying identifier distribution and auxiliary information.

For integrity and freshness, RAI entries are authenticated under R_{RAI}, which is bound to the epoch checkpoint

$$CP_e^A = H(e \parallel R_{\mathrm{RAI}} \parallel v_A \parallel ts_e) \tag{119}$$

An audit response is PQ-signed and contains a query-scoped Merkle multiproof. Modifying a returned entry requires either constructing a false path to R_{RAI} or finding a hash collision. Substituting another audit response requires forging the audit service's PQ signature. Substituting stale state is detected by the epoch, version, timestamp, and blockchain-anchored checkpoint.

Accordingly, the auditor can verify the authenticated evidence and its freshness without disclosure of the private authorization witness or original transaction content. The protocol intentionally reveals query-authorized metadata such as batch identifiers, epochs, versions, and timestamps; these values are therefore outside the privacy claim.

Remark on Query Completeness: A Merkle multiproof establishes membership and integrity of the returned RAI entries but, by itself, does not prove that every entry satisfying an arbitrary semantic predicate has been returned. Therefore, VeRedact-PQ claims authenticated integrity and freshness for the resolved query result. Queries requiring cryptographic result-set completeness additionally require authenticated coverage, range, or non-membership evidence appropriate to the corresponding RAI query structure.

H. Post-Quantum Security and Blockchain Security Boundary

Theorem 8 (Post-Quantum Redaction Security). Under the stated PQ assumptions and provided that the committee and permissioned-consensus thresholds are not violated, a QPT adversary cannot cause an unauthorized redaction to be accepted by VeRedact-PQ except with negligible probability, even when the underlying permissioned blockchain uses classical platform credentials.

Proof: Consider a QPT adversary attempting to create an accepted unauthorized redaction. The VeRedact-PQ acceptance path requires all of the following:

$$\begin{aligned} &\mathsf{PQ\ requester\ authentication} \\ &\Rightarrow \mathsf{PQZK\ policy\ validation} \\ &\Rightarrow t\text{-of-}n\ \mathsf{PQ\ committee\ authorization} \\ &\Rightarrow \mathsf{threshold\ PQCH\ adaptation} \\ &\Rightarrow \mathsf{authenticated\ redaction\ evidence.} \end{aligned} \tag{120}$$

By Theorem 1, bypassing the first two conditions requires breaking PQSIG or PQZK. By Theorem 2, producing committee authorization with fewer than t compromised members requires forging an honest PQ signature, while executing the corresponding state adaptation requires violating threshold PQCH security. Theorems 3–6 prevent a valid request or authorization from being transferred to another batch, state, epoch, or redaction transition. These primitives are assumed secure against QPT adversaries.

Now consider compromise of a classical credential belonging to the underlying Permissioned Blockchain Network (PBN). Such a credential may permit the adversary to authenticate to the platform or submit a native blockchain transaction, depending on the platform's access policy. It does not, however, provide σ_i^R, α_i, the required t committee PQ approvals, or t valid PQCH adaptation shares. Therefore, possession of a classical PBN credential alone cannot satisfy Eq. (120) and cannot produce an accepted VeRedact-PQ redaction.

Likewise, batch checkpoints, audit responses, and setup records are signed under registered PQ keys, so a classical PBN credential cannot forge the anchored state or configuration on which the redaction and audit paths rely.

This establishes a *post-quantum redaction-security boundary*: the security-critical authentication, private policy verification, committee authorization, controlled adaptation, and audit evidence of VeRedact-PQ do not rely solely on the native classical authentication of the PBN.

This result does not imply that the underlying PBN is itself fully post-quantum secure. If a quantum adversary compromises enough native identities to violate the PBN consensus fault-tolerance assumption, ledger integrity or availability may fail independently of the redaction protocol. Such a violation is explicitly outside the threat model. Full post-quantum security of the blockchain substrate would additionally require PQ-secure membership, node authentication, endorsement, transport credentials where applicable, and consensus authentication. Subject to the stated PBN assumption, however, breaking the VeRedact-PQ redaction path requires breaking at least one of its PQ cryptographic assumptions or the committee threshold. Therefore, the probability of an unauthorized accepted redaction is negligible.

V. EVALUATION

This section evaluates VeRedact-PQ through analytical computation- and communication-cost comparison and exper-

TABLE III: Notation Used in the Cost Analysis

Notation	Description
m	Validated requests in one ABRRR batch
$\|\Omega_e\|$	Transaction batches affected by one ABRRR batch
N	Transaction leaves per transaction batch
N_I, N_A	Number of SA-RLI and RAI entries
n, t	Committee size and threshold
n_Q	Redaction records returned by an audit query
$\|\Omega^B_{Q_j}\|$	Distinct authorization batches in an audit result
c	Distinct blocks challenged by an audit in [13] ($c \le n_Q$)
T_H, T_{PRF}	Hash and PRF evaluation
T_S, T_V	PQ signature generation and verification
T_{ZP}, T_{ZV}	PQZK proof generation and verification
T_{CH}	PQCH hash evaluation
T_{PA}, T_{CB}	PQCH partial adaptation and share combination
T^c_V	Classical signature verification
T^c_{AD}, T^c_{CV}	Classical CH adaptation and verification
T^c_{PA}, T^c_{CB}	Classical CH partial adaptation and share combination
T_{Pol}	Policy-based CH authorization (e.g., ABE decryption)
T_{Acc}	RSA-accumulator membership check or update
T_{Tag}	Identity-based RSA tag generation or verification [13]
T^c_E	Classical modular exponentiation

imental performance analysis. Four representative redactable blockchain schemes are used as baselines: Scheme [1] for decentralized threshold redaction with a consistency check; Scheme [13] for auditable redaction with version-freshness proofs under a trusted system manager; Scheme [27] for robust (t, n)-threshold redaction; and Scheme [34] for attribute-based, policy-hiding redaction. Together, these schemes cover complementary capabilities in distributed authorization, policy control, and verifiable auditing, as summarized in Table I. Each baseline enters only the experiments whose stage its paper defines: Schemes [13] and [27] define no authorization step separate from redaction, and Schemes [27] and [34] define no audit protocol.

A. Computation Cost Analysis

This subsection compares the dominant computation costs of VeRedact-PQ with the primary baselines across the four stages of the redaction lifecycle: request validation, authorization, redaction execution, and audit verification. Consider an ABRRR batch containing m validated requests that target $|\Omega_e|$ distinct transaction batches, each committed over N transaction leaves, and an audit query returning $n_Q = |\mathcal{R}_{Q_j}|$ redaction records drawn from $|\Omega^B_{Q_j}|$ distinct authorization batches. The remaining notation is summarized in Table III, and the comparison is presented in Table IV. N/A indicates that the corresponding stage is not explicitly supported by the scheme.

Table IV Table IV shows that the schemes differ mainly in how the cost of authorization, redaction execution, and auditing grows with the number of requests. In Scheme [1], t full nodes each re-check the accumulator and the transaction signatures of every request, and the threshold collision, accumulator update, and Merkle update are repeated per request; moreover, a block can be redacted only once. Scheme [13] avoids authorization altogether by entrusting redaction to a single fully trusted system manager, but every request incurs a chameleon-hash collision, two identity-based tag verifications, a new tag, an accumulator update, and a rebuild of the block's Merkle tree; its audit verifies $c \le n_Q$ challenged blocks with one aggregate decision, so a single invalid record rejects the whole response. Scheme [27] distributes redaction control through threshold adaptation but defines no request validation, approval protocol, or audit, and repeats the t-party adaptation for every request. Scheme [34] enforces a hidden attribute policy through ABE decryption, which, together with the chameleon-hash adaptation, likewise scales with m, and it provides no audit mechanism.

VeRedact-PQ incurs the highest per-request validation cost among the compared schemes because the VPS verifies a PQZK proof T_{ZV} for private policy compliance. This cost is paid once per request: Phase 4 authorization verifies only the compact attestation α_i, and routine audits verify α_i together with the committed $H(\pi_i^{PQ})$ instead of repeating PQZK verification. Committee authorization requires $t(T_S + T_V)$ per ABRRR batch rather than per request, so the committee cost amortized over each request is $t(T_S + T_V)/m$. During execution, BIMC coalesces all modifications that target the same transaction batch, so the number of distributed PQCH adaptations scales with $|\Omega_e|$ rather than m. Since $|\Omega_e| \le m$ and $|\Omega_e| \ll m$ whenever requests cluster on a subset of transaction batches, the most expensive threshold PQ operation is amortized across all co-located modifications. For auditing, committee approvals are verified once per distinct authorization batch, giving $t|\Omega^B_{Q_j}|T_V$ rather than $t\,n_Q T_V$, while the query-scoped multiproof shares authentication paths across returned RAI entries.

Post-quantum primitives are individually more expensive than their classical counterparts. The analytical comparison therefore highlights that VeRedact-PQ concentrates expensive PQ operations at request level (PQZK verification) and batch level (committee authorization and PQCH adaptation), while the per-request online path is dominated by signature verification and hash-based authentication. The experiments below quantify these effects with each baseline on its own classical primitives at about 128-bit security.

B. Communication and Storage Cost Analysis

Let $|R|$, $|\sigma|$, $|\pi|$, and $|h|$ denote the sizes of a redaction request body, a PQ signature, a PQZK proof, and a hash digest, respectively, and let $|\sigma^c|$, $|r^c|$, $|ID|$, $|BH|$, and $|\mathbb{Z}_N|$ denote a classical signature, a classical chameleon-hash randomness, a node identity, a block header, and an RSA-group element. The communication cost of request submission, authorization evidence, on-chain redaction state, and audit responses is summarized in Table V.

Table V shows that post-quantum protection increases the size of individual authentication objects. For example, an ML-DSA-65 signature occupies 3,309 bytes, compared with 64 bytes for a classical Ed25519 signature. Communication efficiency therefore depends on how often such objects are transferred and stored. In VeRedact-PQ, the requester transmits π_i^{PQ} only once to the VPS, after which authorization and routine auditing operate on the attestation α_i and the digest $H(\pi_i^{PQ})$. Committee authorization contributes $t|\sigma|$ per ABRRR batch rather than per request; with $t = 5$ and $m = 64$, for instance, the committee evidence amortized over

TABLE IV: Computation-Cost Comparison of the Redaction Lifecycle

Scheme	Request Validation (per request)	Authorization (per m requests)	Redaction Execution (per m requests)	Audit Verification (per n_Q records)
Scheme [1]	$T_{Acc} + 2T_V^c$	$m\,t(T_{Acc} + 2T_V^c)$	$m(t\,T_{PA}^c + T_{CB}^c + T_{CV}^c + T_{Acc})$ $+O(m \log N)T_H$	$n_Q(T_{CV}^c + T_{Acc})$
Scheme [13]	N/A	N/A	$m(T_{AD}^c + T_{CV}^c + 3T_{Tag} + T_{Acc})$ $+O(mN)T_H$	$O(c)\,T_E^c$
Scheme [27]	N/A	N/A	$m(t\,T_{PA}^c + T_{CB}^c + T_{CV}^c + T_V^c)$	N/A
Scheme [34]	N/A	$m\,T_{Pol}$	$m(T_{AD}^c + 2T_{CV}^c)$	N/A
VeRedact-PQ	$2T_V + T_{ZV} + T_S + T_{\mathrm{PRF}}$ $+O(\log N_I + \log N)T_H$	$m\,T_V + t(T_S + T_V)$ $+O(m)T_H$	$\lvert\Omega_e\rvert(t\,T_{PA} + T_{CB} + T_{CH} + T_S)$ $+O(m(\log N + \log N_I))T_H$	$(n_Q + t\lvert\Omega_{Q_j}^B\rvert + 1)T_V$ $+O(n_Q \log N_A)T_H$

Requester-side proof generation ($T_{ZP} + T_S$) is excluded from request validation. State-transition audits add $O(|\Omega_e|)T_{CH}$; deep audits add $n_Q T_{ZV}$. In [1] a block can be redacted only once and approvals carry no signature; in [27] the t-of-n threshold is enforced only inside Adapt; the audit of [13] returns one decision for all c challenged blocks; the one-off attribute-key issuance of [34] is excluded.

TABLE V: Communication and Storage Cost Comparison

Scheme	Request Submission (per request)	Authorization Evidence (per m requests)	On-Chain Redaction State (per m requests)	Audit Response (per n_Q records)
Scheme [1]	$\lvert R\rvert + \lvert\sigma^c\rvert$	$m\,t\,\lvert ID\rvert$	$m(O(1)\lvert h\rvert + \lvert r^c\rvert)$	$n_Q(\lvert BH\rvert + \lvert\mathbb{Z}_N\rvert)$
Scheme [13]	$\lvert R\rvert$	N/A	$m(\lvert BH\rvert + 3\lvert\mathbb{Z}_N\rvert)$	$c\,\lvert BH\rvert + (c+10)\lvert\mathbb{Z}_N\rvert$
Scheme [27]	N/A	N/A	$m\,\lvert r^c\rvert$	N/A
Scheme [34]	N/A	N/A	$m\,\lvert r^c\rvert$	N/A
VeRedact-PQ	$\lvert R\rvert + \lvert\sigma\rvert + \lvert\pi\rvert$	$t\lvert\sigma\rvert + O(m \log m)\lvert h\rvert$	$\lvert\Omega_e\rvert(\lvert\sigma\rvert + O(1)\lvert h\rvert)$	$(n_Q + t\lvert\Omega_{Q_j}^B\rvert + 1)\lvert\sigma\rvert$ $+O(n_Q \log N_A)\lvert h\rvert$

Deep audits additionally transfer $n_Q|\pi|$; complete PQZK proofs and PBRP objects are held in the off-ledger evidence store. Redacted payloads, which every scheme transfers, are omitted; the $|QR_N|$ elements of [13] are counted as $|\mathbb{Z}_N|$.

each request is approximately 259 bytes instead of 16,545 bytes. Each request additionally carries only an $O(\log m)$ membership proof under R_e^{VR}.

On-chain state grows with the number of affected transaction batches $|\Omega_e|$ rather than the number of executed requests, since Phase 5 anchors one updated checkpoint A_b' per affected batch, while individual redaction records are authenticated through the blockchain-anchored RAI checkpoint. Similarly, audit responses include shared authorization evidence once per distinct batch and a single query-scoped multiproof, avoiding repeated transfer of committee signatures and overlapping Merkle paths. Consequently, VeRedact-PQ compensates for larger PQ objects by transferring and anchoring them at batch granularity.

C. Performance Analysis

This subsection presents the experimental evaluation of VeRedact-PQ under increasing redaction workloads, committee configurations, audit scopes, and on-chain anchoring costs. The objective is to assess whether the proposed batching, coalescing, and cost-aware auditing mechanisms improve end-to-end redaction efficiency while preserving the request-level validation and post-quantum protection defined in Section IV.

1) Experimental Setup: VeRedact-PQ, the four redactable-blockchain baselines [1], [13], [27], [34], and the post-quantum batch authentication scheme of [39] were implemented in approximately 9,900 lines of Python, Rust, and Solidity code and evaluated on an AWS r7i.4xlarge server with an Intel Xeon Platinum 8488C processor (16 vCPUs) and 128 GB RAM, running Ubuntu 24.04. The Besu validator nodes ran as Docker containers on this host, and inter-node latency was set to 10 ms using Linux `tc netem` to emulate a multi-organization deployment; committee members, the VPS, the audit service, and the auditor ran as processes on the same host. In Experiment 1, requesters generated their signatures and PQZK proofs on two separate c7i.48xlarge hosts, so that requester-side proving did not consume the cores of the system under test.

PQSIG was instantiated with ML-DSA-65 (FIPS 204) through the Open Quantum Safe `liboqs` library. H, H_1, H_2, and H_A were instantiated as domain-separated SHA3-256, and PRF as HMAC-SHA3-256 with 256-bit keys K_{idx} and K_A ($\lambda = 256$), consistent with the λ-bit key length of Phase 1. PQZK was instantiated with a transparent hash-based STARK proof system (winterfell 0.13; 42 queries, blowup factor 8, 16-bit grinding), consistent with the trapdoor-free setup assumed in Phase 1. PQCH was implemented as an SIS-based chameleon hash following [10], [17], with distributed adaptation realized through t-of-n trapdoor sharing of a gadget-based lattice trapdoor, dealt without a trusted dealer. Merkle structures, BIMC, SA-RLI, and RAI use binary SHA3-256 trees with multiproof support. The PBN was deployed as a 7-validator Hyperledger Besu network using QBFT consensus with a 2-s block period, with Solidity contracts anchoring policy commitments, committee configurations, batch checkpoints, and RAI checkpoints; PQ signature verification is performed by the validator nodes before anchoring rather than inside the contracts. Table VI reports the measured execution time of each primitive.

The workload consists of synthetic enterprise transactions

TABLE VI: Execution Time of Cryptographic Primitives

Operation	Instantiation	Time (ms)
VeRedact-PQ		
T_S / T_V	ML-DSA-65 sign / verify	0.085 / 0.032
T_{ZP} / T_{ZV}	PQZK prove / verify	22.898 / 0.722
T_{CH}	PQCH hash	1.196
T_{PA} / T_{CB}	PQCH partial adapt / combine	18.835 / 5.820
T_H	SHA3-256 (64-byte input)	0.001
T_{PRF}	HMAC-SHA3-256	0.004
Baselines		
T_V^c	ECDSA secp256k1 verify [1], [27]	0.028 / 0.027
T_{PA}^c / T_{CV}^c	Improved DCH, BLS12-381 [1]	0.666 / 2.838
T_{PA}^c / T_{CV}^c	ETCH, secp256k1 [27]	0.038 / 0.057
T_{AD}^c / T_{CV}^c	Double-trapdoor CH, RSA-3072 [13]	7.989 / 8.516
T_{AD}^c / T_{CV}^c	CHET, RSA-3072 [34]	56.813 / 50.662
T_{Acc}	RSA-3072 accumulator [1], [13]	0.760
T_{Tag}	Identity-based RSA-3072 tag [13]	62.187 / 54.689
T_{Pol}	MA-ABE decryption, BLS12-381 [34]	7.716
T_E^c	RSA-3072 exponentiation [13]	7.761
Post-quantum baseline		
T_m^L	Polynomial multiplication in R_q [39]	0.265
T_S^L / T_{ZV}^L	Lattice sign + ZKP / ZKP verify [39]	1.031 / 0.725
T_{AG}^L / T_{AV}^L	Aggregate / verify, 64 signatures [39]	13.745 / 20.169

TABLE VII: Default Experimental Parameters

Parameter	Default value
Committee size n / threshold t	[7] / [5]
ABRRR bounds $B_{\min}$ / $B_{\max}$	[8] / [256]
Maximum waiting time $T_{\max}$	[500] ms
Transaction leaves per batch N	[256]
SA-RLI shards S / RAI shards S_A	[16] / [16]
Ledger size	[10^6] transactions
Zipf skew s	[0.8]
Signature scheme	ML-DSA-65 (NIST category 3)
Runs	1 per point; 30 samples per point (Exp. 2–4), 5 at $n_Q = 10^5$

with payloads of 256 B to 4 KB, organized into transaction batches of N leaves. Redaction targets follow a Zipf distribution with skew s to control batch locality, where $s = 0$ corresponds to uniformly distributed targets and larger s concentrates requests on fewer transaction batches. Request arrivals follow a Poisson process whose rate is varied per experiment, and every scheme replays the same request trace generated from one seed. Unless otherwise stated, the parameters in Table VII are used. Each configuration point is executed once; reported values are medians (95th percentiles for latency) over all requests or samples of that run, and primitive timings are medians with 95% confidence intervals.

To separate architectural effects from primitive choice, all baselines were reimplemented within the same framework and instantiated with their own classical primitives at about 128-bit security, since none of their papers defines a post-quantum variant. As a post-quantum reference point, Scheme [39] (LETA) is also included: it authenticates a batch of requests with module-lattice signatures and a per-requester zero-knowledge proof, which a leader aggregates and a single verifier checks. It was instantiated on the ML-DSA ring ($n = 256$, $q = 8{,}380{,}417$) with module rank 5, the security level of VeRedact-PQ. Since it defines no redaction, audit, or on-chain step, it enters only Table VI and Experiment 2. Stages not defined by a baseline's paper are reported as not supported. Each baseline otherwise follows its original workflow, including per-request authorization and adaptation. For auditing, Scheme [1] holds its accumulator membership proofs locally, an option its paper provides, and the per-record accumulator witnesses of Scheme [13] are timed individually on one core and summed. The redaction histories audited in Experiments 3 and 4 are built beforehand and are not timed. Since the paper of Scheme [1] fixes no block size and its chain permits one redaction per block, each of its blocks holds one transaction, so that its history reaches the largest audit point.

Experiment 1: Redaction Latency and Throughput: This experiment evaluates end-to-end redaction performance as the request arrival rate increases from 100 to 5,000 requests/s over the default ledger, comparing VeRedact-PQ with Refs. [1], [13], [27], and [34]. End-to-end latency is measured from request arrival to blockchain finalization of the updated checkpoint, covering Phases 3 to 5, over every request that arrives in the measurement window; a request not finalized by the end of a 30-s drain period is counted with its elapsed time at that moment, so a point with such requests is a lower bound, and throughput is measured as the number of redactions finalized per second within a 60-s window that follows a 10-s warm-up. The results are shown in Fig. 3.

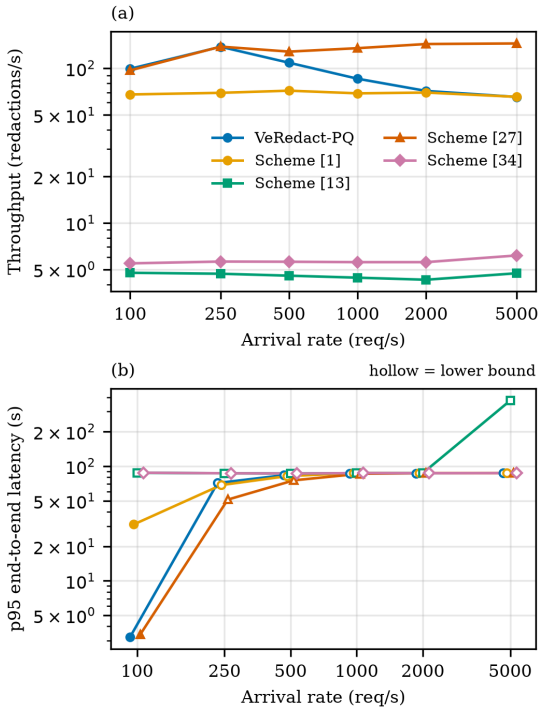


Fig. 3: Redaction performance: (a) throughput and (b) 95th-percentile end-to-end latency versus request arrival rate (hollow markers: lower bounds).

Fig. 3(a) shows that Ref. [27] achieves the highest throughput, about 128–145 redactions/s from 250 requests/s onward, consistent with its low-cost classical adaptation ($T_{PA}^c = 0.038$ ms per redactor in Table VI). VeRedact-PQ finalizes 99 and 138 redactions/s at 100 and 250 requests/s and 65–109 redactions/s at higher rates, whereas each of its distributed PQCH adaptations requires post-quantum partial adaptations ($T_{PA} = 18.8$ ms per member). Ref. [1] reaches 65–72 redactions/s; its chain additionally permits only one redaction per block, so requests to an already redacted block are rejected.

Refs. [13] and [34] saturate at about 5 and 6 redactions/s, consistent with the RSA-3072 tag and CHET operations of about 57–62 ms each that every redaction requires.

Fig. 3(b) reports the 95th-percentile end-to-end latency on a logarithmic scale. At 100 requests/s, every request of VeRedact-PQ and Ref. [27] is finalized, with 3.2 and 3.4 s, respectively, and Ref. [1] requires 31.0 s, whereas Refs. [13] and [34] finalize none of the requests arriving in the window before the drain ends, so their latency is at least 87 s at every rate. At 250 requests/s, VeRedact-PQ, Ref. [27], and Ref. [1] exceed their throughput and leave 29%, 40%, and 70% of the requests unfinished, giving lower bounds of 71.3, 51.1, and 68.4 s, respectively. From 500 requests/s onward all schemes are saturated and their latency approaches the observation limit of about 87 s. Overall, the per-adaptation cost of post-quantum primitives keeps VeRedact-PQ below the lightweight classical threshold scheme of Ref. [27], while it sustains higher throughput than Ref. [1] up to 2,000 requests/s and more than ten times that of Refs. [13] and [34].

Experiment 2: Transaction Authorization Latency: This experiment evaluates the complete authorization latency, i.e., the Phase 3 validation of each request together with the Phase 4 multi-party authorization of its batch, as the committee size increases as $n \in \{4, 7, 10, 16, 32\}$ with $t = \lfloor 2n/3 \rfloor + 1$, and the ABRRR batch size as $m \in \{1, 8, 32, 64, 128, 256\}$. Phase 4 is measured from batch closure to the availability of $Auth_e^B$, and the latency is reported per request, as the Phase 3 validation plus the Phase 4 batch latency divided by m, and per batch of m requests. VeRedact-PQ is compared with Scheme [1] and Scheme [34], which support distributed or policy-based redaction authorization but define no batch protocol, so a batch of m requests costs m of their per-request authorizations, and with the post-quantum batch authentication of Scheme [39], whose latency covers the per-request identity check, aggregation of the m signatures, and verification of the aggregate; Schemes [13] and [27] define no separate authorization step. Each configuration is measured over 30 authorization batches. The results are shown in Fig. 4.

Fig. 4(a) shows that the per-request authorization latency of VeRedact-PQ decreases from 3.25 ms at $m = 1$ to 1.70 ms at $m = 8$ and 1.41 ms at $m = 256$, because the t committee signatures and their verification are shared by all m requests; the remaining 1.3 ms is the Phase 3 validation of each request, which verifies its ML-DSA-65 signature and PQZK proof and signs its validation attestation. Schemes [1] and [34] have no batch protocol, so their latency stays at 0.47 and 7.2 ms for every m. Scheme [1] is faster because its t approving nodes only check the accumulator and verify the classical ECDSA signatures of the transaction, without a policy or requester proof, whereas Scheme [34] is 5.1 times slower than VeRedact-PQ at $m = 256$ because every request requires an attribute-based decryption. The post-quantum Scheme [39] decreases from 1.55 to about 0.57 ms; it verifies the requesters' lattice signatures in aggregate but performs no policy-compliance or state-freshness check, which accounts for its lower latency.

Fig. 4(b) shows the latency of one batch of $m = 64$ requests. It increases with committee size for the committee-based schemes, from 91 to 102 ms for VeRedact-PQ, from

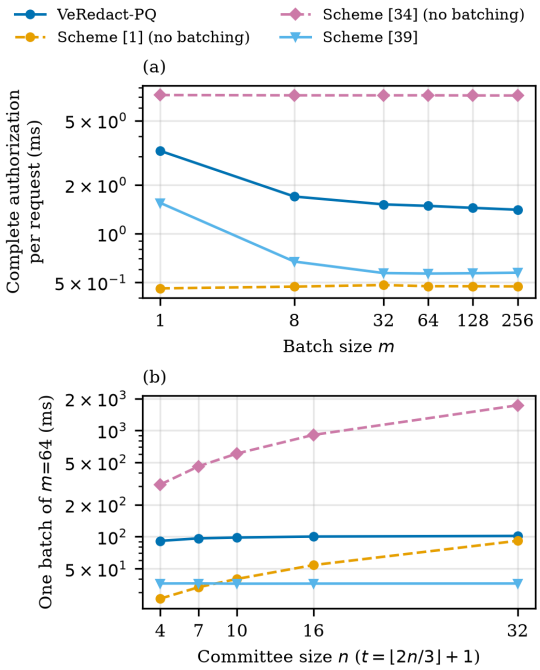


Fig. 4: Authorization latency (per-request validation and batch authorization): (a) per request versus batch size and (b) per batch of $m = 64$ requests versus committee size. Dashed: schemes without a batch protocol.

26 to 91 ms for Scheme [1], and from 310 to 1,726 ms for Scheme [34] as n grows from 4 to 32, while Scheme [39], which has a single verifier and no committee, stays at about 36 ms. For VeRedact-PQ, the committee term is incurred once per ABRRR batch and therefore affects each request only through the amortized term $t(T_S + T_V)/m$; its latency grows by 11% from $n = 4$ to $n = 32$, compared with 3.5 and 5.6 times for Schemes [1] and [34]. Consequently, larger committees, which strengthen distributed control, remain practical under high redaction volumes.

Experiment 3: Audit Efficiency: This experiment evaluates the service-side efficiency of Phase 6 auditing, namely query resolution, evidence generation, and response size. The number of returned redaction records is varied as $n_Q \in \{10, 10^2, 10^3, 10^4, 10^5\}$, with records drawn from authorization batches of 64 requests. VeRedact-PQ is compared with Scheme [13] and Scheme [1]. Response-generation time and response size are measured, each over 30 queries (5 at $n_Q = 10^5$, and 5 for Scheme [13] at $n_Q = 10^4$). The results are shown in Fig. 5.

Fig. 5(a) shows that VeRedact-PQ generates audit responses fastest, from 0.38 ms at $n_Q = 10$ to 57 ms at $n_Q = 10^4$ and 443 ms at $n_Q = 10^5$, because records are resolved through the sharded RAI and the query-scoped multiproof $MP_{Q_j}^A$ shares internal Merkle nodes. Ref. [1] requires 3.3 ms to 27.7 s, and Ref. [13] requires 11 min to 73 h, since it builds one RSA-accumulator non-membership witness per returned record.

Fig. 5(b) shows the opposite ordering for response size. VeRedact-PQ returns the largest responses, from 61 KB to 395 MB, about 4 KB per record, dominated by the

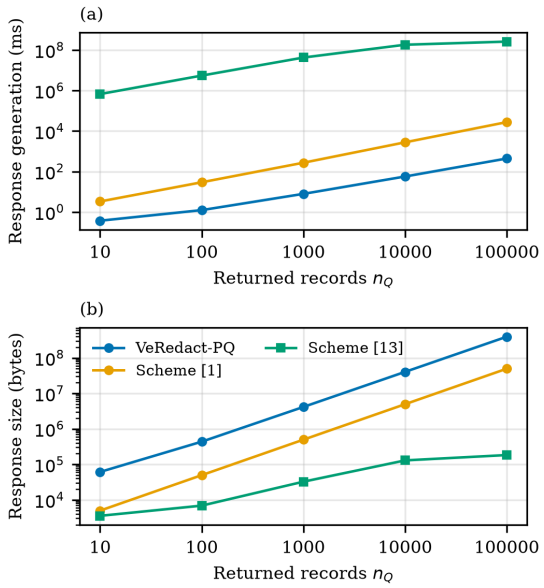


Fig. 5: Audit efficiency: (a) response-generation time and (b) audit-response size versus the number of returned records.

3,309-byte ML-DSA-65 validation attestation α_i carried by each record, whereas the shared authorization evidence $(R_e^{VR}, C_e^B, Auth_e^B)$ is included only once per distinct authorization batch. Ref. [1] returns 5.0 KB to 49.6 MB, and Ref. [13], which aggregates its proof, returns 3.5 to 183 KB. Overall, the results show that sharded RAI resolution and query-scoped evidence keep audit-service computation low, while post-quantum signatures dominate the communication cost of record-level evidence.

Experiment 4: Verification Time: This experiment evaluates auditor-side verification of returned redaction evidence. The number of verified records is varied as in Experiment 3 for the **Normal Audit**, which verifies the response signature and query binding, the RAI multiproof, committee approvals, validation attestations, and the state-transition and PQCH checks of every record. VeRedact-PQ is compared with Refs. [13] and [1] in their original classical instantiations. To evaluate verification granularity, modified, substituted, and stale evidence is additionally injected into 1%, 5%, and 10% of returned records, and the fraction of invalid records individually rejected and valid records retained is measured. The results are shown in Fig. 6.

Fig. 6 shows that normal-audit verification time of VeRedact-PQ grows approximately linearly with n_Q, from 6.7 ms at $n_Q = 10$ to 9.4 s at $n_Q = 10^4$, 3.5 to 4.8 times faster than Ref. [1] (32 ms to 32.6 s). Ref. [13] verifies one aggregated proof in a nearly constant 53–59 ms and is therefore faster for large queries. At $n_Q = 10^4$, the state-transition and PQCH checks account for 8.1 s of the 9.4 s, while committee approvals contribute only 51 ms ($t|\Omega_{Q_j}^B|T_V$); routine audits therefore rely only on PQ-signed attestations and hash-based authentication, and complete PQZK verification is reserved for challenged or forensic audits. Because each returned record carries individual membership and execution evidence, VeRedact-PQ and Ref. [1] reject every injected modified, substituted, or stale record individually while retaining all valid records, as stale evidence fails the version, epoch, and checkpoint checks of Phase 6. In contrast, the aggregated decision of Ref. [13] rejects the whole response, including all valid records, whenever a single record is invalid.

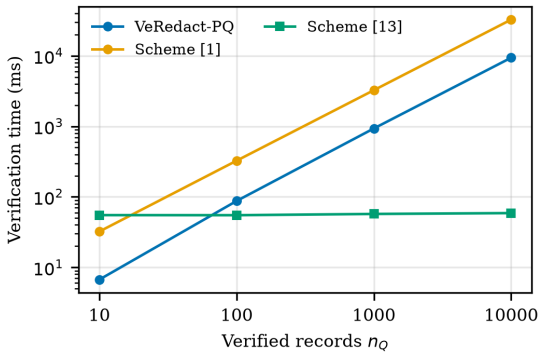


Fig. 6: Verification time versus number of verified records for the normal audit.

Experiment 5: Blockchain Gas Consumption: This experiment evaluates the on-chain cost of VeRedact-PQ on the permissioned Besu network. Although the gas price is set to zero in the permissioned deployment, gas usage provides a deterministic measure of on-chain computation and storage. The measured contract operations are policy and committee registration (Phase 1), batch-checkpoint anchoring A_b (Phase 2), authorization anchoring $H(Auth_e^B)$ (Phase 4), redaction finalization, which atomically commits the updated checkpoints $\{A_b'\}$ and SA-RLI state (Phase 5), and RAI checkpoint anchoring CP_e^A (Phase 6). Every scheme finalizes the same 1,024 requests under Zipf skew $s = 0.8$. The baselines were implemented as contracts that store the on-chain redaction state prescribed by each scheme; since none of them defines batching, each redaction is its own transaction. The gas per finalized redaction is compared across schemes, with VeRedact-PQ both without batching ($m = 1$) and at its default ABRRR batch size ($m = 64$), and the effect of the batch size $m \in \{1, 8, 32, 64, 128, 256\}$ is evaluated for VeRedact-PQ alone. The results are shown in Fig. 7.

Fig. 7(a) shows that the baselines consume 119,261 to 153,825 gas per redaction, since each redaction updates its own on-chain state in one transaction; Scheme [1] is the most expensive because each redacted block header also records its updated accumulator, which its auditor reads. Without batching, VeRedact-PQ consumes 160,127 gas per redaction, more than every baseline, because each request additionally anchors its authorization $H(Auth_e^B)$. At the default batch size of 64, it consumes 81,039 gas, 32% less than the cheapest baseline, Scheme [27]. Fig. 7(b) shows that VeRedact-PQ becomes cheaper than Scheme [27] from $m = 8$ (96,294 gas) and reaches 70,727 gas at $m = 256$, because one authorization anchor serves all m requests and redaction finalization commits one updated checkpoint per affected transaction batch $|\Omega_e|$ rather than per request, while individual redaction records are authenticated through the RAI checkpoint.

Table VIII further reports the gas consumed by each contract

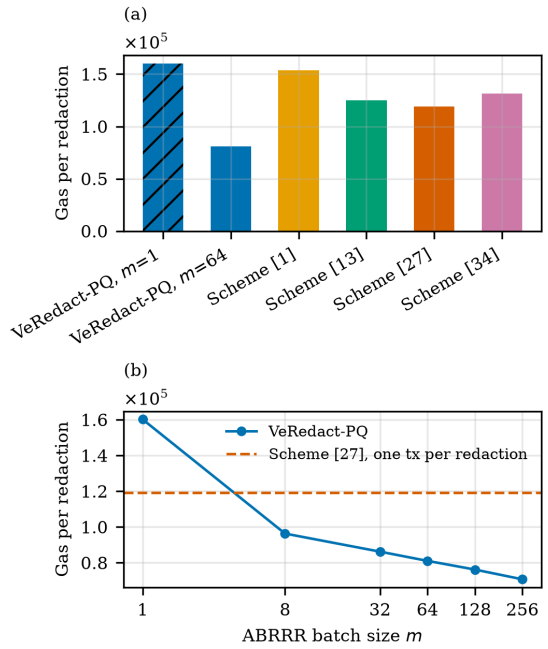


Fig. 7: Gas consumption: (a) gas per finalized redaction of each scheme and (b) gas per redaction of VeRedact-PQ versus the ABRRR batch size, with the cheapest baseline as reference.

TABLE VIII: Gas Consumption of VeRedact-PQ Contract Operations

Operation	Frequency	Gas
Policy registration	Per policy version	[TBD]
Committee registration	Per epoch	[TBD]
Batch checkpoint A_b	Per transaction batch	[TBD]
Authorization anchor $H(Auth_e^B)$	Per ABRRR batch	44,460
Redaction finalization $\{A'_b\}$	Per ABRRR batch	115,645 ($\|\Omega_e\| = 1$)
RAI checkpoint CP_e^A	Per audit epoch	45,811

operation. Overall, the results show that VeRedact-PQ bounds on-chain cost by the number of batch-root transitions and the audit-epoch frequency, rather than by the total redaction volume.

VI. Acknowledgement

This research has been supported by the Thammasat University Research Unit in Cyber Security.

References

[1] C. Li, Q. Shen and Z. Wu, "Redactable Blockchain From Decentralized Chameleon Hash Functions, Revisited," in IEEE Transactions on Computers, vol. 74, no. 6, pp. 1911-1920, June 2025, doi: 10.1109/TC.2025.3544878.

[2] K. Huang, X. Zhang, Y. Mu, X. Wang, G. Yang, X. Du, F. Rezaeibagha, Q. Xia, and M. Guizani, "Building Redactable Consortium Blockchain for Industrial Internet-of-Things," *IEEE Trans. Ind. Informat.*, vol. 15, no. 6, pp. 3670–3679, Jun. 2019, doi: 10.1109/TII.2019.2901011.

[3] W. Wang, L. Wang, J. Duan, X. Tong, and H. Peng, "Redactable Blockchain Based on Decentralized Trapdoor Verifiable Delay Functions," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7492–7507, 2024, doi: 10.1109/TIFS.2024.3431917.

[4] H. Guo, W. Gan, M. Zhao, C. Zhang, T. Wu, L. Zhu, and J. Xue, "PriChain: Efficient Privacy-Preserving Fine-Grained Redactable Blockchains in Decentralized Settings," *Chinese Journal of Electronics*, vol. 34, no. 1, pp. 82–97, 2025, doi: 10.23919/cje.2023.00.305.

[5] D. Zhang, J. Le, X. Lei, T. Xiang, and X. Liao, "Secure Redactable Blockchain With Dynamic Support," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 2, pp. 717–731, 2024, doi: 10.1109/TDSC.2023.3261343.

[6] X. Wu, X. Du, Q. Yang, N. Wang, and W. Wang, "Redactable consortium blockchain based on verifiable distributed chameleon hash functions," *J. Parallel Distrib. Comput.*, vol. 183, Art. no. 104777, 2024, doi: 10.1016/j.jpdc.2023.104777.

[7] S. Aguincha, E. Nunes, S. Eisa, and M. L. Pardal, "ChainGuards: Verification of Sensed Data using Permissioned Blockchain Technology," arXiv preprint arXiv:2603.20769, 2026.

[8] D. Derler, K. Samelin, D. Slamanig, and C. Striecks, "Fine-Grained and Controlled Rewriting in Blockchains: Chameleon-Hashing Gone Attribute-Based," in *Proc. 26th Annu. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2019, doi: 10.14722/ndss.2019.23066.

[9] G. Ateniese, B. Magri, D. Venturi, and E. R. Andrade, "Redactable Blockchain – or – Rewriting History in Bitcoin and Friends," in *Proc. 2nd IEEE Eur. Symp. Security Privacy (EuroS&P)*, Paris, France, pp. 111–126, 2017, doi: 10.1109/EuroSP.2017.37.

[10] C. Peng, H. Xu, and P. Li, "Redactable Blockchain Using Lattice-based Chameleon Hash Function," in *Proc. 2022 Int. Conf. Blockchain Technology Information Security (ICBCTIS)*, pp. 94–98, 2022, doi: 10.1109/ICBCTIS55569.2022.00032.

[11] C. Wu, L. Ke, and Y. Du, "Quantum resistant key-exposure free chameleon hash and applications in redactable blockchain," *Information Sciences*, vol. 548, pp. 438–449, 2021, doi: 10.1016/j.ins.2020.10.008.

[12] K. Y. Chan, L. Chen, Y. Tian, and T. H. Yuen, "Reconstructing Chameleon Hash: Full Security and the Multi-Party Setting," in *Proc. 19th ACM Asia Conf. Computer and Communications Security (AsiaCCS)*, pp. 1076–1091, 2024, doi: 10.1145/3634737.3656291.

[13] X. Zhang, Z. Cai, K. Chen, G. Ha and C. Jia, "Efficient Auditing and Querying in Verifiable Redactable Blockchain: A Lightweight VDS Protocol with Integrity Verification," 2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), Guiyang, China, 2025, pp. 504-511, doi: 10.1109/Trustcom66490.2025.00062.

[14] K. Huang, X. Zhang, Y. Mu, F. Rezaeibagha, and X. Du, "Scalable and redactable blockchain with update and anonymity," *Information Sciences*, vol. 546, pp. 25–41, 2021, doi: 10.1016/j.ins.2020.07.016.

[15] Y. Dong, Y. Li, Y. Cheng, and D. Yu, "Redactable consortium blockchain with access control: Leveraging chameleon hash and multi-authority attribute-based encryption," *High-Confidence Computing*, vol. 4, no. 1, Art. no. 100168, 2024, doi: 10.1016/j.hcc.2023.100168.

[16] Y. Zhang, Z. Ma, S. Luo, and P. Duan, "Dynamic Trust-Based Redactable Blockchain Supporting Update and Traceability," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 821–834, 2024, doi: 10.1109/TIFS.2023.3326379.

[17] X. Wang, Y. Chen, X. Zhu, C. Li, and K. Fang, "A Redactable Blockchain Scheme Supporting Quantum-Resistance and Trapdoor Updates," *Applied Sciences*, vol. 14, no. 2, Art. no. 832, 2024, doi: 10.3390/app14020832.

[18] J. B. Klamti and M. A. Hasan, "Revocable policy-based chameleon hash using lattices," *Journal of Mathematical Cryptology*, vol. 18, no. 1, Art. no. 20230012, pp. 152–182, 2024, doi: 10.1515/jmc-2023-0012.

[19] X. Huang, Y. Wang, Y. Ding, Q. Wu, C. Yang, and H. Liang, "Dynamically redactable Blockchain based on decentralized Chameleon hash," *Digital Communications and Networks*, vol. 11, no. 3, pp. 757–767, 2025, doi: 10.1016/j.dcan.2024.10.013.

[20] M. Miao, X. Yang, J. Wei, G. Tian, and W. Susilo, "A Verifiable and Redactable Blockchain with Lightweight Storage and Permission Supervision," *Information*, vol. 17, no. 2, Art. no. 176, 2026, doi: 10.3390/info17020176.

[21] S. Fugkeaw, S. Sungchai, S. Nakprame, and P. Sreekongpan, "Enabling Secure and Scalable GDPR-Compliant Blockchain-Based e-KYC With Efficient Redaction," *IEEE Access*, vol. 13, pp. 136834–136853, 2025, doi: 10.1109/ACCESS.2025.3594656.

[22] C. G. Harris, "A Redactable Blockchain Architecture for Regulatory Compliance," 2025 IEEE International Conference on Blockchain (Blockchain), Zhengzhou, China, 2025, pp. 1-6, doi: 10.1109/Blockchain67634.2025.00010.

[23] Z. Wu, L. Wang, X. Zhang and X. Feng, "EMT: Extended Merkle Tree Structure for Inserted Data Redaction in Permissioned Blockchain," in IEEE Transactions on Network Science and Engineering, vol. 12, no. 4, pp. 3025-3038, July-Aug. 2025, doi: 10.1109/TNSE.2025.3555979.

[24] J. W. Heo, G. Ramachandran and R. Jurdak, "Decentralised Redactable Blockchain: A Privacy-Preserving Approach to Addressing Identity Tracing Challenges," 2024 IEEE International Conference on Blockchain

and Cryptocurrency (ICBC), Dublin, Ireland, 2024, pp. 215-219, doi: 10.1109/ICBC59979.2024.10634438.

[25] T. Sengupta, S. Chakraborty, and S. Sural, "ReAcct: Redaction control for interoperable blockchains," in *Proc. 7th Conf. Blockchain Research Appl. Innovative Networks Services (BRAINS)*, Zurich, Switzerland, 2025, pp. 1–10, doi: 10.1109/BRAINS67003.2025.11302947.

[26] L. Yin, J. Shi, B. Xie and Y. Liu, "Rchain: A Universal and Efficient Redactable Blockchain Scheme," 2025 IEEE International Conference on Blockchain (Blockchain), Zhengzhou, China, 2025, pp. 235-240, doi: 10.1109/Blockchain67634.2025.00038.

[27] Z. Liu, B. Zhou and Y. Zhao, "Enhancing Redactable Blockchain With Robust and Efficient Threshold Redaction," in IEEE Transactions on Dependable and Secure Computing, vol. 23, no. 2, pp. 3238-3251, March-April 2026, doi: 10.1109/TDSC.2025.3634512.

[28] F. Wang, R. Dong, J. Cui, Q. Zhang and H. Zhong, "Blockchain-Based Anonymous and Accountable Data Redaction for the Industrial Internet of Things," in IEEE Transactions on Cloud Computing, doi: 10.1109/TCC.2026.3725253.

[29] S. Biswas, S. Chakraborty, and R. S. Chakraborty, "Redacting without regret: A dual-layer framework for policy-compliant blockchains," in *Proc. 7th Conf. Blockchain Research Appl. Innovative Networks Services (BRAINS)*, Zurich, Switzerland, 2025, pp. 1–9, doi: 10.1109/BRAINS67003.2025.11302930.

[30] C. Li, Q. Shen and Z. Wu, "Redactable Blockchain From Decentralized Chameleon Hash Functions, Revisited," in IEEE Transactions on Computers, vol. 74, no. 6, pp. 1911-1920, June 2025, doi: 10.1109/TC.2025.3544878.

[31] Z. Yuqi, "Policy-Hiding Chameleon Hash with Traceability for Redactable Blockchains," 2025 22nd International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP), Chengdu, China, 2025, pp. 1-4, doi: 10.1109/ICCWAMTIP68645.2025.11352637.

[32] S. Hu, M. Li, J. Weng, J. -N. Liu, J. Weng and Z. Li, "IvyRedaction: Enabling Atomic, Consistent and Accountable Cross-Chain Rewriting," in IEEE Transactions on Dependable and Secure Computing, vol. 21, no. 4, pp. 3883-3900, July-Aug. 2024, doi: 10.1109/TDSC.2023.3339675.

[33] L. Xue, H. Huang, F. Xiao, Q. Li and W. Wang, "A Controllable, Publicly Auditable, and Redactable Blockchain With a Main–Auxiliary Architecture," in IEEE Transactions on Dependable and Secure Computing, vol. 23, no. 2, pp. 1847-1864, March-April 2026, doi: 10.1109/TDSC.2025.3620854.

[34] J. Xue et al., "Attribute-Based Policy-Hiding Redactable Blockchain With Authorizable Verification for Energy Internet," in IEEE Internet of Things Journal, vol. 12, no. 15, pp. 29570-29583, 1 Aug.1, 2025, doi: 10.1109/JIOT.2025.3569699.

[35] K. Huang, X. Li, F. Rezaeibagha, L. Zhang and X. Zhang, "Time Updatable Policy-Based Chameleon Hash for Traceable and Accountable Redactable Blockchain," in IEEE Transactions on Information Forensics and Security, vol. 21, pp. 1470-1483, 2026, doi: 10.1109/TIFS.2026.3655919.

[36] J. Duan, W. Wang, L. Wang and L. Gu, "Controlled Redactable Blockchain Based on T-Times Chameleon Hash and Signature," in IEEE Transactions on Information Forensics and Security, vol. 19, pp. 7560-7572, 2024, doi: 10.1109/TIFS.2024.3436925.

[37] W. -C. Kuo, D. -R. Lin and S. -X. Chen, "Identity-Based Chameleon Hashing from the SIS Assumption in a Central Authority Model," 2025 IEEE Conference on Dependable and Secure Computing (DSC), Taipei, Taiwan, 2025, pp. 1-6, doi: 10.1109/DSC65356.2025.11260874.

[38] M. Shirmohammadi, A. Islam and H. Karimipour, "An Intent-Based Networking Framework for Secure and Privacy-Compliant Machine Unlearning Using Meta-Learning and Redactable Blockchain," in IEEE Internet of Things Journal, vol. 13, no. 9, pp. 18167-18181, 1 May1, 2026, doi: 10.1109/JIOT.2025.3638966.

[39] Q. Li, Y. Xia, X. Liu, X. Chen, Z. Liu, Z. Guo, Z. Liu and L. Fang, "LETA: A Lattice-Based Efficient and Traceable Privacy-Preserving Batch Authentication Scheme for Vehicle Platoon in VANETs," in IEEE Journal on Selected Areas in Communications, vol. 44, pp. 2186-2200, 2026, doi: 10.1109/JSAC.2025.3636982.